

ASIC physical implementation with Synopsys tools

Turning the Ibex RISC-V processor from code into a chip layout

This is a companion to Miguel's Cadence student tutorial. It follows the same stages with the Synopsys tools. The later stages take a whole chip on IHP's kit through finishing, testing and release packaging. Every example comes from a recorded run. The repository `ibex-sg13g2-chip` holds the chip's scripts, so you can build the same chip yourself.

Prepared by Cesar Valentin. Every command, screenshot and number comes from the run `tutorial_ibex_saed32` unless the text names another run. We started that run on October 1, 2026 at 01:19.

Evidence and scope as of October 5, 2026. Stages 1 to 4 show the SAED32 block runs from October 1. The checker those runs used judges them. Stages 5 to 7 show `test1`, a fresh build of the `ibex-sg13g2-chip` repository from a clean clone on October 4 and 5. Its final signoff says `SIGNOFF CLEAN` with 139 of 139 checks passing on October 5, 2026. IHP is the factory. IHP can accept the design only after we submit it.

Item	Value
Document	Ibex tutorial: RTL to GDSII with Synopsys
Version	2.0, 2026-10-05
Author	Cesar Valentin
Audience	team members new to ASIC physical design
Repository	https://github.com/XxMasterpicxX/ibex-sg13g2-chip
Block run	<code>tutorial_ibex_saed32</code> , stages 1 to 4, a worked example to read
Chip run	<code>test1</code> , a fresh build of the repository, stages 5 to 7
Chip result	<code>SIGNOFF CLEAN</code> with 139 of 139 checks passing on October 5, 2026
Status	Released for team use

Contents

Introduction.....	4
From code to chip.....	4
Conventions.....	5
Glossary.....	5
Timing and load.....	8
Miguel's tools and ours.....	8
Before you start.....	10
Logging in.....	10
Getting the repository.....	10
The ~/flash folder.....	11
The design.....	12
Step 0: make a run folder.....	12
Step 1: run it.....	14
Stage 1: Synthesis.....	15
1.1 Read the code.....	15
1.2 Set up the corners and the timing rules.....	15
1.3 Build it from cells.....	16
1.4 Check the result.....	16
Stage 2: Physical design.....	18
2.1 Floorplan.....	18
2.2 Power grid.....	19
2.3 Placement.....	21
2.4 Clock tree.....	23
2.5 Routing.....	24
2.6 Finish and write the files.....	26
2.7 Timing and area at each step.....	27
Stage 3: Signoff.....	28
3.1 Wire delays, StarRC.....	28
3.2 Timing, PrimeTime SI.....	28
3.3 Same logic as the code, Formality.....	29
3.4 Power, PrimePower.....	30
3.5 Factory rules and connections, IC Validator.....	30
3.6 The final answer, check.py.....	31
3.7 Supply voltage drop, RedHawk-SC.....	31
The fix loop.....	33
ECO rounds.....	33
The last few by hand.....	35
End of the block run.....	36
Mistakes we made on the block.....	37
Stage 4: The final files.....	38
We ran it twice.....	39

Stage 5: Making it a chip.....	40
5.1 The chip.....	40
5.2 Pads, seal ring and fill.....	41
5.3 Checking the whole chip.....	43
Stage 6: Proving it works.....	45
6.1 Timing at every corner, in both modes.....	45
6.2 Adding timing margin.....	46
6.3 Running real programs on the gates.....	47
6.4 Testing for factory defects.....	49
6.5 Signals from the board.....	49
6.6 Supply voltage drop and electromigration.....	50
Stage 7: Ready for the factory.....	52
7.1 Review records.....	52
7.2 The release package.....	53
7.3 Mistakes we made on the chip.....	54
7.4 Status of the chip.....	55
One-page summary.....	56
References.....	57

Introduction

From code to chip

A digital chip starts as code that engineers write in a hardware language such as Verilog. That code is called RTL, short for register transfer level. The name says what the code describes: registers that store values, and the logic that moves data between them on each tick of the clock. The factory cannot build code. It needs a drawing of every transistor and wire on every layer. That drawing is a GDSII file, which everyone shortens to GDS. The work of turning RTL into GDS is physical implementation. This tutorial does all of it.

The work happens in seven stages. In stage 1, synthesis turns the code into a list of small logic blocks called cells. In stage 2, physical design places those cells on the chip and wires them together. Both stages run in Fusion Compiler. That is the Synopsys tool that builds the design. In stage 3, signoff checks the result with separate tools. Stage 4 freezes the final files. Each stage ends with a check. Do not move on until it passes.

Stages 1 to 4 build one block, the Ibex processor. They use SAED32, a teaching kit from Synopsys. A teaching kit has no factory behind it. That is why stages 5 to 7 switch to IHP SG13G2. It is the free kit of a real factory in Germany. On that kit the same flow builds a whole chip. The chip adds two memories, pads, a seal ring and metal fill around Ibex. Stages 6 and 7 check it the way a team checks a real chip before it goes to the factory.

Ibex on SAED32, RTL to GDSII, as this tutorial runs it

Two commands drive it: `run_fc.sh` for stages 1 and 2, `signoff.sh` for stage 3.

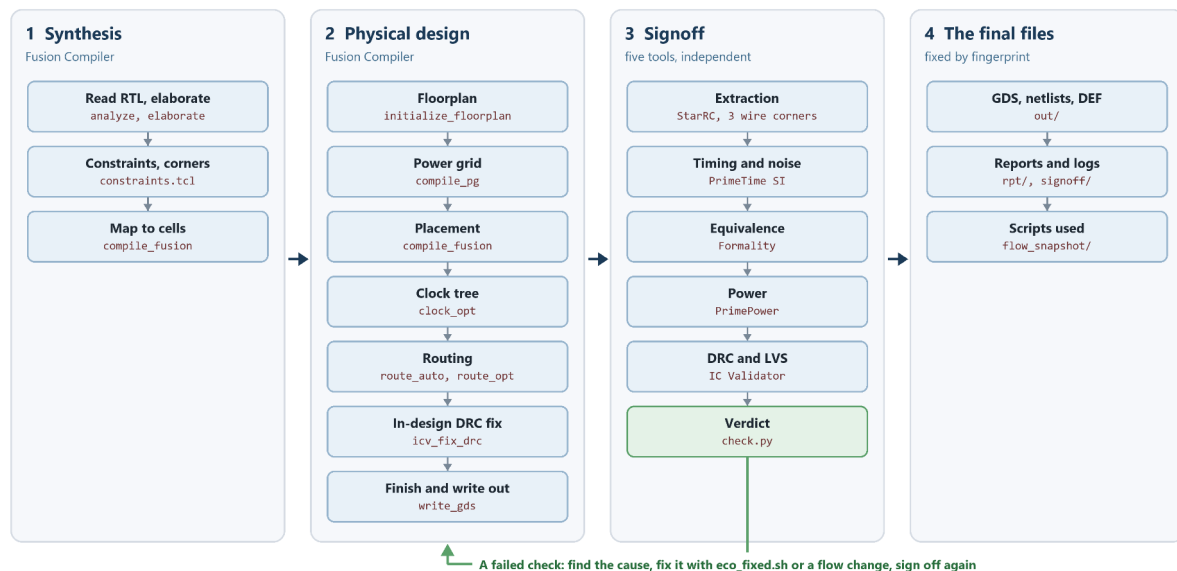


Figure 1. Stages 1 to 4 build and check one block. The script `run_fc.sh` runs stages 1 and 2 in one Fusion Compiler session. The script `signoff.sh` runs stage 3. It runs five separate checking tools and then `check.py`. The green arrow at the bottom is the fix loop. A failed check sends the work back to stage 2.

From a signed-off block to a chip a factory could build

The same Synopsys flow, now on IHP's open kit, with the checks a real tapeout needs.

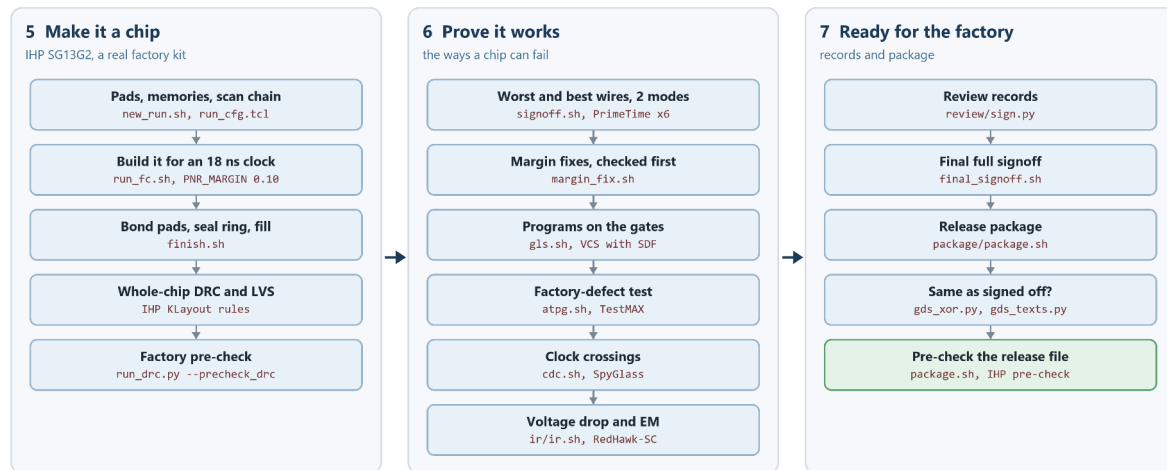


Figure 2. Stages 5 to 7 turn a block into a chip a factory could build. Each box names one step. The line under it names the script or tool that does it.

Every terminal screenshot shows a real command and its real output. Some commands pick out or shorten report lines so they fit. Stages 1 to 4 are a worked example. Their SAED32 kit settings are not in the repository, so read them for the ideas and the checks. Stages 5 to 7 you can repeat. Each one gives the repository command that does that step. Our own run used earlier versions of the same scripts, so a few screenshots show older script names.

Conventions

You see	It means
a grey box in Conso1as	a command or a piece of a script. Commands run on the server from a terminal
~	your home folder on the server. The repository is cloned into ~/flash, and the runs, scripts and reports all live there
\$RUN	the run folder. Step 1 sets it for the block and section 5.1 for the chip. Run later boxes in the same terminal so they reuse it
... is done when	the check that must pass before you move on
Figure N	a flow diagram, a real screenshot or a layout picture from the run. The numbers go up in order through the document
ns, mV, μ m, fF, mW	nanoseconds, millivolts, micrometers, femtofarads and milliwatts

Read stages 1 to 4 in order the first time. They teach the flow on one block. Stages 5 to 7 assume you know them and only explain what is new for a whole chip. The one-page summary at the end lists each step and its main command. Use it as a quick reminder.

Glossary

You will see these words all through the tutorial. Come back here when one is unclear.

Word	What it means
RTL	register transfer level. The chip described as code, here in Verilog
standard cell	a small ready-made logic block from the library, like an AND gate or an inverter. Each is a few transistors wired as one simple function. Think of them as LEGO bricks
flip-flop	a cell that stores one bit. It takes a new value each time the clock ticks
clock	the signal that ticks at a fixed rate. The SAED32 block's clock ticks every 5 ns, 200 million times a second. The IHP chip's clock ticks every 20 ns
netlist	the list of cells and the wires that connect them. Each wire is a net. Section 1.4 shows one line of a real netlist
timing path	the route a signal takes from one flip-flop through logic cells to the next flip-flop. The timing tools check one path at a time. An endpoint is the pin where a path ends. Figure 3 draws one
pin	a point where a wire connects to a cell, or to the edge of the block
slack	the time to spare on a path. Positive slack passes. Negative slack fails
setup	a signal must arrive a little before the next clock tick. Too slow is a setup failure. Figure 3 shows it on a timeline
hold	a signal must not arrive too soon after a tick. Too fast is a hold failure
corner	a set of conditions the chip must work in. Transistors are slowest at low voltage and high temperature. That is the slow corner. They are fastest at high voltage and low temperature. That is the fast corner
PDK	process design kit, which this tutorial calls the kit. It comes from the factory and holds the cell library, the layer rules and the models
block	one finished piece of a chip. In this tutorial the block is the Ibex processor alone
core	the inner area of the block where the cells go
row	cells sit side by side in straight rows of equal height, like bricks in a wall
metal layers	a chip has many layers of wiring stacked above the transistors. M1 is the lowest. Higher numbers are higher up
routing	drawing the metal wires that connect the cells
short	two wires touch that should not
open	a wire that should connect two points has a gap
buffer	a cell that passes a signal along unchanged. It has strong transistors, so it can charge a long wire or many inputs quickly. Figure 4 shows why that matters
stronger cell	a bigger version of the same cell, such as INVX8 in place of INVX4. Its bigger transistors charge its load faster
crosstalk	when a signal switching on one wire disturbs the wire next to it. It can slow down or speed up the neighbor's signal. It can also put a small false pulse on it. That pulse is noise

Word	What it means
load	everything a cell's output has to charge up. That is the wire and the inputs of the cells it feeds. More load makes the signal slower. Load is measured in femtofarads, written fF. One fF is a millionth of a billionth of a farad. The limits in this tutorial run from 8 fF for a small cell to 64 fF for a clock buffer
electrical limits	the most load each cell may drive, and how slowly its signal may rise or fall
via	a small metal plug that joins one metal layer to the next
DRC	design rule check. It asks whether every shape is wide enough and far enough from its neighbors for the factory to make it
LVS	layout versus schematic. It asks whether the drawing connects the same circuit as the netlist
GDS	the final layout file the factory builds the chip from
ECO	engineering change order. A small targeted change after routing, such as making one cell stronger
signoff	the final checks. Separate tools that did not build the design run them. The script check.py reads their reports and gives one verdict
checkpoint	a point where the Fusion Compiler script saves the design. A run can stop or start again there
pad	a large metal square on the chip's edge where a wire from the package attaches
seal ring	a metal wall around the chip's edge that keeps out cracks and moisture
fill	metal shapes that carry no signal. The flow adds them so every layer meets the factory's density rules
SRAM	a ready-made memory block from the factory. This tutorial calls them memories. Our chip has two, for program and data
scan chain	a test mode that links every flip-flop into one long chain. A tester loads and reads the whole chain through two pins
ATPG	automatic test pattern generation. It makes the patterns a factory tester uses to find broken chips
SDF	a file of every cell and wire delay. A simulation that reads it runs at real speed
clock crossing, CDC	a signal that the chip's clock does not time, such as one from the test board. CDC stands for clock domain crossing. A single wire like that passes through a synchronizer before the logic uses it. A synchronizer is two or three flip-flops in a row. A group of wires that must change together needs more than that
IR drop	how much the supply voltage sags on its way through the power grid to each cell
EM	electromigration. Too much current through a thin wire wears it away over years
review record	a short signed note in which an engineer accepts something a tool cannot judge on its own. The note includes the evidence
shuttle	a factory run that builds many teams' small chips on the same wafer to share the cost

Timing and load

Most of the checks in this tutorial come back to two ideas. A signal must arrive at the right time, and a cell must be strong enough for what it drives. The next two pictures show both. They explain ideas, so their shapes are drawn by hand and do not come from a run.

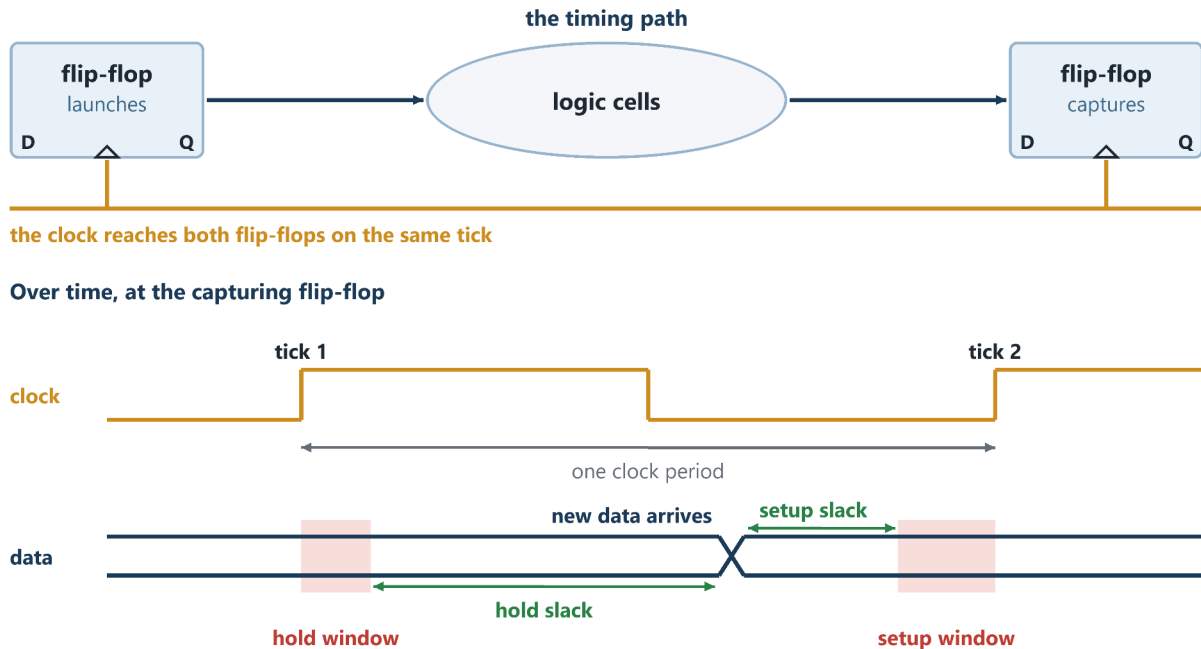


Figure 3. A timing path runs from one flip-flop, through logic cells, to the next flip-flop. The data must arrive before the setup window of the next tick. It must also stay away from the hold window right after a tick. Slack is the time to spare.

Setup and hold pull in opposite directions. A slow path risks setup, so the slow corner checks it hardest. A fast path risks hold, so the fast corner checks that. Making a slow path faster can make a short path too fast. That is why every fix in this tutorial is checked at all three corners.

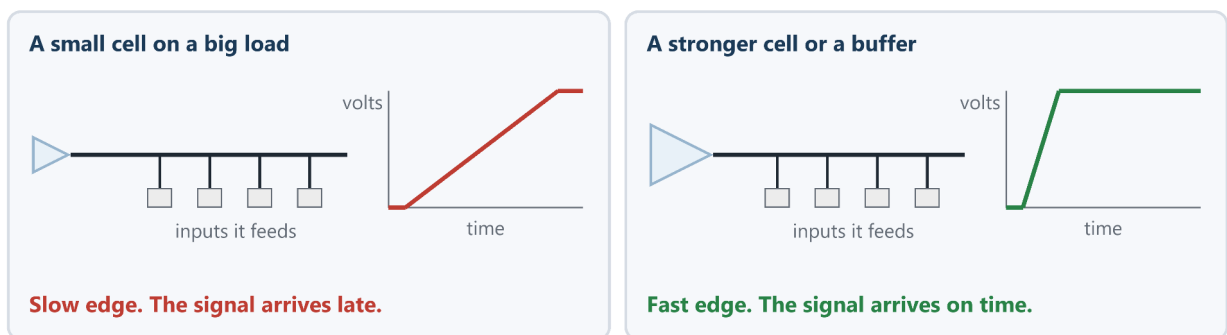


Figure 4. A cell has to charge its load, the wire and every input it feeds. A small cell on a big load makes a slow edge. A stronger cell or a buffer makes a fast one.

Miguel's tools and ours

The stages are the same as in Miguel's tutorial. Only the tool names change.

Job	Miguel's tutorial, Cadence	This tutorial, Synopsys	In our flow
Synthesis	Genus	Fusion Compiler	<code>compile_fusion</code> in <code>fc_flow.tcl</code>
Place and route	Innovus	Fusion Compiler	same session as synthesis
Wire delays	Quantus	StarRC	3 wire corners, in <code>signoff.sh</code>
Timing	Tempus	PrimeTime SI	3 corners, with crosstalk
Power	Voltus	PrimePower	typical corner
Voltage drop	Voltus	RedHawk-SC	static drop, section 3.7
DRC and LVS	Pegasus	IC Validator	the SAED32 rule files
Equivalence	not covered	Formality	netlist against RTL
Chip DRC and LVS	Pegasus	KLayout with IHP's rules	IHP ships no IC Validator rules, stages 5 and 6
Programs on the gates	Xcelium	VCS	with SDF delays, stage 6
Factory-defect test	Modus	TestMAX ATPG	stage 6
Clock crossings	Conformal CDC	SpyGlass CDC	stage 6
Electromigration	Voltus	RedHawk-SC	IHP's published limits, stage 6

Where this tutorial differs from Miguel's, and why:

- Synthesis and place and route happen in the same tool on the same design. Nothing passes from one tool to another in between.
- There is no separate step for fixing timing before the clock tree. The placement command already does it.
- The wire delays we trust come from StarRC. It is a separate tool that we run after routing. Fusion Compiler only estimates them while it works.
- We add an equivalence check with Formality. It proves the final netlist does the same logic as the RTL. A design can pass every timing and DRC check and still be the wrong circuit.
- The script `check.py` reads the reports and gives one verdict. We call it the checker. It works from a fixed list of checks called its contract. SIGNOFF CLEAN means every check in that list passed. It does not mean we ran every check the kit could need.

Before you start

Logging in

- Log in to a Linux server where the Synopsys and Ansys tools live under /apps. Off campus, connect to the VPN first. Without it, ssh cannot find the server.
- Our scripts load the tools for you. If you start a tool by hand, run `source /apps/settings` first. Otherwise the tool will say it has no license.
- Fusion Compiler licenses are shared with other users. When no license is free, `run_fc.sh` tries again.

Getting the repository

Clone the repository into ~/flash and run its setup once. Every script finds its files in ~/flash, so do not clone it anywhere else.

```
git clone https://github.com/XxMasterpicxX/ibex-sg13g2-chip.git ~/flash
bash ~/flash/setup/setup.sh
```

Setup takes about two hours. It downloads IHP's kit, builds KLayout, and installs sv2v and the Python packages. It then builds the libraries and wire models the Synopsys tools read, from IHP's kit and with your own licenses. Those files are not in the repository, because they are in Synopsys formats. Setup is done when ~/flash/setup/setup.log ends with SETUP_DONE, after one line ending in _OK for each step. If it stops, the last line names the step, and that step's log says why. Run `setup.sh` again after fixing it. Every step that already passed skips itself.

```

user@server: ~/flash

[user@server flash]$ ls
chip
designs
flow
ihp
LICENSE
NOTICE
pdk
pdk_cfg
README.md
runs
setup
tools
[user@server flash]$ tac setup/setup.log | sed '/^SETUP_START/q' | tac | grep -E "^(SETUP_|[A-Z0-9_]+_OK)" | sed
"s#$_HOME/$_#~#"
SETUP_START 2026-10-04 23:05:39
PDK_OK 5e6d592e4002946a4616f798c357f0f3c06cf3b6
FILL_MACRO_OK ~/flash/ihp/fill/sg13g2_filler_Metal_mid.lym
SRAM_LEF_OK
SV2V_OK sv2v v0.0.13
PYENV_OK klayout 0.30.5
KLAYOUT_OK KLayout 0.30.5
DB_OK 12 libraries
NDM_OK
RC_OK
KIT_OK
SETUP_DONE 2026-10-04 23:05:48
[user@server flash]$

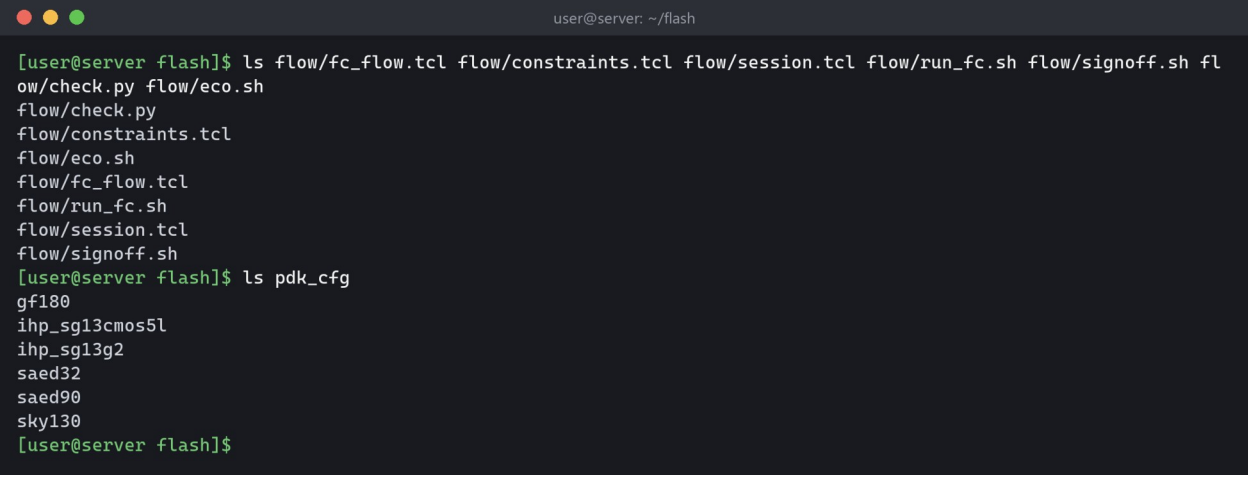
```

Figure 5. The repository after setup, and the end of setup.log from a fresh clone in a test home. This was a second run of setup.sh. Every step had passed before, so each one only checked its result.

The ~/flash folder

Path	What it holds
flow/fc_flow.tcl	the Fusion Compiler script that takes the design from RTL to GDS. It has eight checkpoints
flow/constraints.tcl	the clock and timing rules that Fusion Compiler and PrimeTime both read
flow/run_fc.sh	runs the flow for one run folder
flow/signoff.sh	runs all the checking tools, then check.py
flow/check.py	reads every report and gives the final answer
flow/eco_fixed.sh	the fix loop: PrimeTime proposes or tests changes and Fusion Compiler applies them
pdk_cfg/ihp_sg13g2/	every fact about the IHP kit: layers, cells, corners, pads and fill
designs/flash_soc/	the chip's RTL, testbench and test programs
chip/	the chip's own scripts: the one-command build, margin rounds, programs on the gates, scan test, clock crossings, voltage drop, review records and the release package
ihp/	the scripts that build the Synopsys libraries, and the wire models
setup/	the one-time setup
runs/<name>/	one folder per run. Each holds its settings, design, reports and outputs

The flow scripts never name a layer, a cell or a rule. All of that comes from `pdk_cfg/<kit>/pdk.tcl`. That is why the same flow ran on SAED32 for the block and on IHP for the chip. The repository holds only the IHP settings.

A terminal window with a dark background and light green text. The window title is 'user@server: ~/flash'. The first command is `ls flow/fc_flow.tcl flow/constraints.tcl flow/session.tcl flow/run_fc.sh flow/signoff.sh flow/check.py flow/eco.sh`, which lists several files in the 'flow' directory. The second command is `ls pdk_cfg`, which lists kit settings: `gf180`, `ihp_sg13cmos5l`, `ihp_sg13g2`, `saed32`, `saed90`, and `sky130`.

```
[user@server flash]$ ls flow/fc_flow.tcl flow/constraints.tcl flow/session.tcl flow/run_fc.sh flow/signoff.sh flow/check.py flow/eco.sh
flow/check.py
flow/constraints.tcl
flow/eco.sh
flow/fc_flow.tcl
flow/run_fc.sh
flow/session.tcl
flow/signoff.sh
[user@server flash]$ ls pdk_cfg
gf180
ihp_sg13cmos5l
ihp_sg13g2
saed32
saed90
sky130
[user@server flash]$
```

Figure 6. The flow scripts and the kit settings in our own work folder.

The design

The design is Ibex, a small 32-bit RISC-V processor made by lowRISC. We took it from its GitHub page at version e9f5534 and converted it into one Verilog file, `designs/ibex_sv2v/ibex_sv2v.v`. Its top module is `ibex_top`. The top module is the outermost block that holds all the others. Ibex is big enough to show real problems with wiring, timing and the clock. It is also small enough that the build and the checks take about 30 minutes.

The kit is Synopsys SAED32, a teaching kit that models a 32 nm process. We use it because it ships IC Validator rule files. Those files keep every step on Synopsys tools. SAED90 is also on the server and has them too.

Step 0: make a run folder

A run is a folder in `~/flash/runs` with one settings file in it, `run_cfg.tcl`. OUT in that file must be the full path of the folder. The figure shows the whole settings file of our block run. The table explains each setting. For the chip, `chip/new_run.sh` makes the folder and the file for you.

```

user@server: ~/flash/runs/tutorial_ibex_saed32

[user@server tutorial_ibex_saed32]$ cat run_cfg.tcl
set DESIGN      ibex_top
set RTL_FILES    [list ~/flash/designs/ibex_sv2v/ibex_sv2v.v]
set INC_DIRS     {}
set DEFINES      {}
set CLK_PORT     clk_i
set PERIOD_NS    5.0
set PNR_MARGIN   0.03
set UTIL         0.60
set MAX_LAYER    M6
set PDK          saed32
set OUT          ~/flash/runs/tutorial_ibex_saed32
[user@server tutorial_ibex_saed32]$

```

Figure 7. The run settings. OUT must point at the run folder itself. run_fc.sh will not start if it points anywhere else.

Setting	Value	What it means
DESIGN	ibex_top	the top module
RTL_FILES	ibex_sv2v.v	the code to read
CLK_PORT	clk_i	the clock input
PERIOD_NS	5.0	one clock tick every 5 ns, which is 200 MHz
PNR_MARGIN	0.03	0.03 means 3%. Fusion Compiler aims at a clock 3% shorter, 4.85 ns, so the real 5 ns has room to spare
UTIL	0.60	cells fill 60% of the area at the start. The rest is room for wires
MAX_LAYER	M6	signal wires may use metal layers up to M6
PDK	saed32	which kit settings to load
OUT	the run folder	where the run writes everything

The block has to work at all three corners below. The kit settings pick a voltage, a temperature and a matching cell library for each corner. Each corner also has a wire corner. The wire corner sets how much the wires slow signals down. The column "Mainly for" names the main check for each corner. PrimeTime still checks setup and hold at every corner. Commands and reports call the typical corner typ. The kit sets these values. Together they cover the range of voltage and temperature a real chip could meet.

Corner	Voltage	Temperature	Mainly for	Wire corner
slow	0.95 V	125 C	setup	most wire delay
typ	1.05 V	25 C	power	typical
fast	1.16 V	-40 C	hold	least wire delay

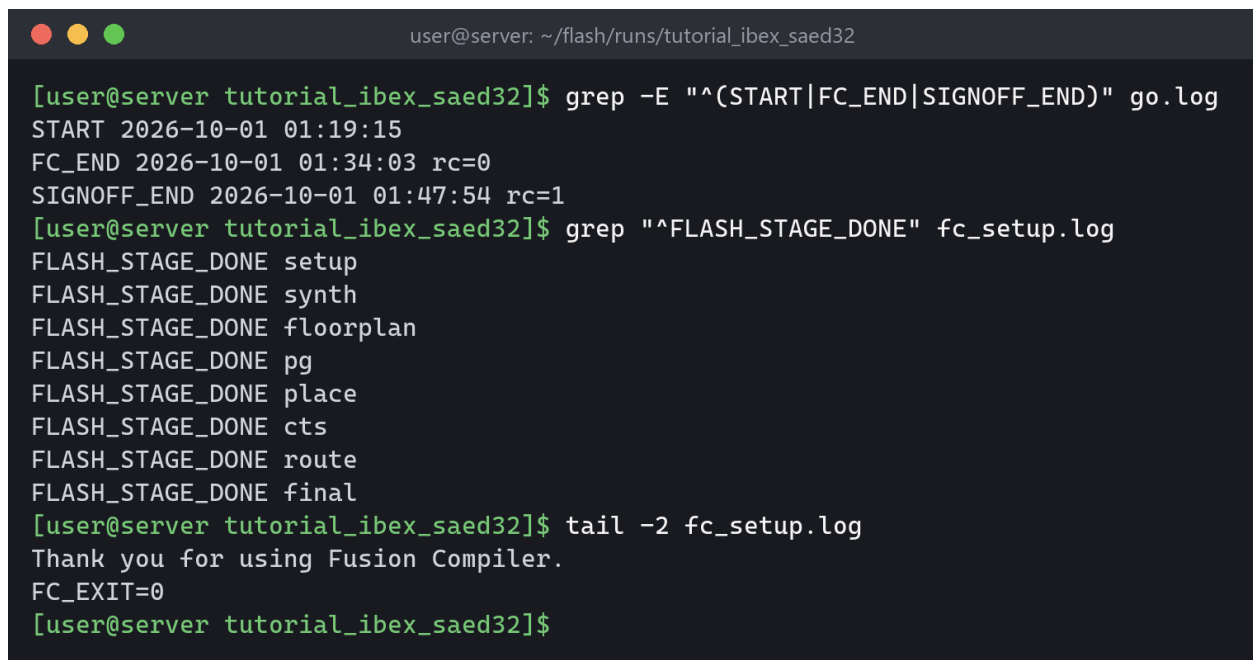
Step 1: run it

Two commands run everything. The first one runs Fusion Compiler from RTL to GDS. The second one runs signoff, the final checks. Both run in the background with `nohup`, so they keep going if you log out. Start the second one only after `fc_setup.log` has the line `FLASH_FLOW_DONE`. That file is the run's log.

Fusion Compiler always ends its log with `FC_EXIT=0`, even when the script stopped on an error. A run only worked if its log has the line `FLASH_FLOW_DONE`. `run_fc.sh` checks this for you.

```
RUN=~/.flash/runs/tutorial_ibex_saed32
nohup ~/flash/flow/run_fc.sh $RUN > $RUN/fc.out 2>&1 &
# after that one finishes
nohup ~/flash/flow/signoff.sh $RUN > $RUN/signoff.out 2>&1 &
```

In our run, Fusion Compiler took 14.8 minutes and signoff took 13.9 minutes. The Fusion Compiler script has eight checkpoints. At each one it saves the design and writes `FLASH_STAGE_DONE` and the checkpoint name to `fc_setup.log`.

A terminal window with a dark background and light green text. The prompt is 'user@server: ~/flash/runs/tutorial_ibex_saed32'. The user enters 'grep -E "(START|FC_END|SIGNOFF_END)" go.log' and the output shows 'START 2026-10-01 01:19:15', 'FC_END 2026-10-01 01:34:03 rc=0', and 'SIGNOFF_END 2026-10-01 01:47:54 rc=1'. Then the user enters 'grep "FLASH_STAGE_DONE" fc_setup.log' and the output lists eight checkpoints: 'FLASH_STAGE_DONE setup', 'FLASH_STAGE_DONE synth', 'FLASH_STAGE_DONE floorplan', 'FLASH_STAGE_DONE pg', 'FLASH_STAGE_DONE place', 'FLASH_STAGE_DONE cts', 'FLASH_STAGE_DONE route', and 'FLASH_STAGE_DONE final'. Finally, the user enters 'tail -2 fc_setup.log' and the output shows 'Thank you for using Fusion Compiler.' and 'FC_EXIT=0'.

```
user@server: ~/flash/runs/tutorial_ibex_saed32
[user@server tutorial_ibex_saed32]$ grep -E "(START|FC_END|SIGNOFF_END)" go.log
START 2026-10-01 01:19:15
FC_END 2026-10-01 01:34:03 rc=0
SIGNOFF_END 2026-10-01 01:47:54 rc=1
[user@server tutorial_ibex_saed32]$ grep "FLASH_STAGE_DONE" fc_setup.log
FLASH_STAGE_DONE setup
FLASH_STAGE_DONE synth
FLASH_STAGE_DONE floorplan
FLASH_STAGE_DONE pg
FLASH_STAGE_DONE place
FLASH_STAGE_DONE cts
FLASH_STAGE_DONE route
FLASH_STAGE_DONE final
[user@server tutorial_ibex_saed32]$ tail -2 fc_setup.log
Thank you for using Fusion Compiler.
FC_EXIT=0
[user@server tutorial_ibex_saed32]$
```

Figure 8. Following the run, `go.log` shows when Fusion Compiler and signoff ended. `fc_setup.log` has one `FLASH_STAGE_DONE` line for each of the eight checkpoints. It still ends with `FC_EXIT=0`. Signoff ended with `rc=1`, an error code. In this run the code means the checker found problems. Stage 3 covers them.

The eight checkpoints are `setup`, `synth`, `floorplan`, `pg`, `place`, `cts`, `route` and `final`. You do not have to rerun the whole flow after a change to a later step. To start again from one checkpoint, set `START_AT`. To stop after one, set `STOP_AFTER`.

```
START_AT=route ~/flash/flow/run_fc.sh $RUN # start again from routing
STOP_AFTER=place ~/flash/flow/run_fc.sh $RUN # stop once placement is saved
```

Stage 1: Synthesis

Synthesis reads the RTL and builds the same logic out of standard cells from the library. It uses the setup and synth checkpoints of `fc_flow.tcl`. The first checkpoint is named setup because it sets up the design. It has nothing to do with the setup timing check.

Goes in	Comes out
the Verilog code, <code>ibex_sv2v.v</code>	a netlist of library cells, inside the design
the clock and timing rules, <code>constraints.tcl</code>	an area report, <code>rpt/area_synth.rpt</code>
the cell libraries for 3 corners	a timing report, <code>rpt/timing_synth.rpt</code>

The code boxes in stages 1 and 2 show short parts of `flow/fc_flow.tcl` so you can see what the script does. Open that file in the repository for the whole script. You do not need to type them. The two commands in Step 1 run all of it.

1.1 Read the code

The script makes a new design library, reads the code and builds the design. If reading fails, the script stops the run on purpose. That way it can never go on with old code by mistake.

```
create_lib $OUT/$DESIGN.nlib -use_technology_lib [lindex $NDM_LIB 0] -ref_libs $NDM_LIB
if {[analyze -format sverilog -define $DEFINES $RTL_FILES]} {
    puts "FLASH_ERROR analyze failed"; exit 1
}
if {[elaborate $DESIGN]} { puts "FLASH_ERROR elaborate failed"; exit 1 }
set_top_module $DESIGN
```

1.2 Set up the corners and the timing rules

Next the script sets the supply voltage, the temperature and the process number of each corner. For each corner it makes a scenario. A scenario is one set of timing checks. Every scenario reads the same `constraints.tcl` that PrimeTime reads later. Both tools therefore check the same rules.

```
foreach {c v t db} $CORNERS {
    create_corner $c
    set_voltage $v -corners $c -object_list [get_supply_nets VDD]
    set_temperature $t -corners $c
    set_process_number [dict get $CORNER_PROCESS $c] -corners $c
    create_scenario -name func_$c -mode func -corner $c
    current_scenario func_$c
    source $FLOW/constraints.tcl
}
```

The timing rules in `constraints.tcl` for this run:

- The clock ticks every 4.85 ns in Fusion Compiler and every 5 ns in PrimeTime. The difference comes from the `PNR_MARGIN` setting in Step 0.
- The script assumes the circuits outside our block use 1 ns of each 5 ns tick. That leaves a signal entering or leaving the block about 4 ns inside it.
- Each path keeps 0.25 ns spare for setup so that small errors do not break the chip.
- In signoff, PrimeTime adds a 5% delay penalty. For setup it makes the launching clock and data path 5% slower and the capturing clock path 5% faster. For hold it does the reverse. This covers small differences between transistors on a real chip. PrimeTime calls this a derate. Section 3.2 explains it.

Each library has a process number that tells how fast its transistors are. In SAED32 the slow library is 0.99, the typical one is 1.00 and the fast one is 1.01. Fusion Compiler picks each corner's library by this number, the voltage and the temperature. An early version of our script gave every corner the number 1. Fusion Compiler then used the typical library everywhere. PrimeTime later found the slow corner failing.

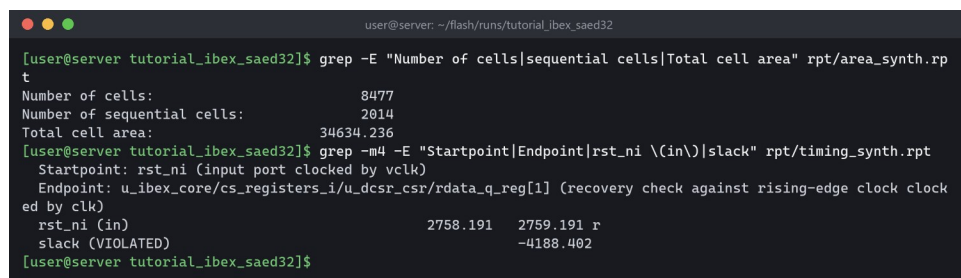
This part of the script also turns on the crosstalk check. In an early run without it, Fusion Compiler said `lbex` passed. PrimeTime then found it failing by 1.47 ns.

1.3 Build it from cells

One command maps the logic onto library cells and then improves it.

```
compile_fusion -to logic_opto
```

1.4 Check the result



```
user@server: ~/flash/runs/tutorial_ibex_saed32
[user@server tutorial_ibex_saed32]$ grep -E "Number of cells|sequential cells|Total cell area" rpt/area_synth.rpt
Number of cells: 8477
Number of sequential cells: 2014
Total cell area: 34634.236
[user@server tutorial_ibex_saed32]$ grep -m4 -E "Startpoint|Endpoint|rst_ni |(in\\)|slack" rpt/timing_synth.rpt
Startpoint: rst_ni (input port clocked by vclk)
Endpoint: u_ibex_core/cs_registers_i/u_dcsr_csr/rdata_q_reg[1] (recovery check against rising-edge clock clock
ed by clk)
rst_ni (in) 2758.191 2759.191 r
slack (VIOLATED) -4188.402
[user@server tutorial_ibex_saed32]$
```

Figure 9. Synthesis results from `rpt/area_synth.rpt` and `rpt/timing_synth.rpt`. Check the cell count and area first. The worst timing path starts at the reset input `rst_ni`.

Synthesis made 8,477 cells. Of these, 2,014 work with the clock. Almost all of those are flip-flops. Together the cells cover 34,634 μm^2 .

The timing looks awful, with -4,188 ns of slack. Read the path before you worry. It starts at the reset input `rst_ni`, and that one pin takes 2,758 ns. Reset goes to almost every flip-flop, and right now a single wire drives all of them. The tool sees one enormous slow wire. That delay is far longer than a whole 5 ns tick, which tells you it is not a real problem yet. Placement adds a tree of buffers to drive it. After placement, the worst slack is +0.004 ns.

The netlist is plain text. This is one real line of it, from the finished design in out/ibex_top.v:

```
AND2X1_RVT A7629 ( .A1 ( N16782 ) , .A2 ( N16776 ) , .Y ( N4255 ) ) ;
```

It places one cell. AND2X1_RVT is the cell type from the library, a small two-input AND gate. A7629 is the name of this copy of it. The inputs A1 and A2 connect to the nets N16782 and N16776. The output Y drives the net N4255. The netlist has one entry like this for every cell in the design.

Read the worst path before you decide what a slack number means. In this run the reset wire causes the large delay. After synthesis, check the cell count and the area. Use the timing only to spot paths with too many logic cells in a row.

Stage 1 is done when the script read the code with no errors and the cell count and area look right. The worst paths must also be ones that placement will fix.

Stage 2: Physical design

Physical design turns the netlist into a layout. First it sets the size of the block and builds the power grid. Next it places every cell and builds the clock tree. Last it draws every wire. Each step depends on the one before it. This stage runs these checkpoints of `fc_flow.tcl`: `floorplan`, `pg`, `place`, `cts`, `route` and `final`.

Goes in	Comes out
the netlist from stage 1	the finished design, saved as <code>ibex_top/final</code>
the cell layouts from the kit	<code>out/ibex_top.gds</code> , the layout
the power grid recipe from <code>pdk.tcl</code>	<code>out/ibex_top.v</code> , the netlist for timing checks
routing layers M2 to M6	<code>out/ibex_top.pg.v</code> , the netlist with power connections for LVS
	<code>out/ibex_top.def</code> , the placed and routed design for StarRC

2.1 Floorplan

The script works out the size from the cell area and the 60% target set by UTIL. It leaves an 8 μm border for the power ring. It spreads the pins around all four sides on metal layers M3 and M4.

```
initialize_floorplan -core_utilization $UTIL -core_offset $CORE_OFFSET -shape R
set_block_pin_constraints -self -allowed_layers $PIN_LAYERS -sides {1 2 3 4}
place_pins -self
```

The block is 256.16 by 255.10 μm , about a quarter of a millimeter on each side. The script placed all 930 pins. Check that the line `FLASH_UNPLACED_PORTS` in the log says 0. In an old run, one unplaced clock pin turned off the whole clock tree step. Fusion Compiler printed no error.

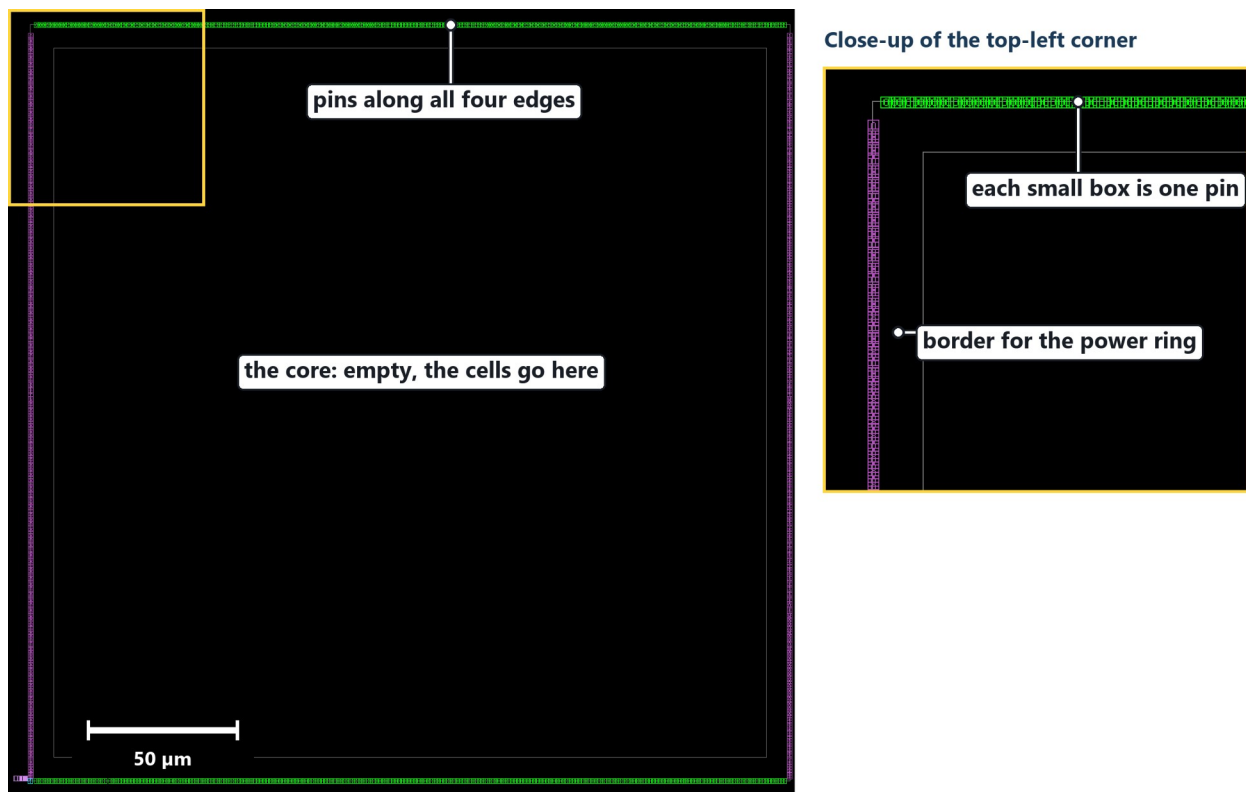


Figure 10. The floorplan. The core in the middle is still empty. The 930 pins sit along the edges, and the close-up shows them as small boxes. The border between the pins and the core is room for the power ring.

The floorplan is done when the size makes sense for the area and no pin is left unplaced.

2.2 Power grid

Every cell needs a power wire and a ground wire. We call the power wire VDD and the ground wire VSS. The grid has four parts. A ring circles the core, and a mesh of wide stripes covers it on M6 and M7. Thin straps sit on M2 below the mesh. Rails on M1 run along every row of cells. The mesh reaches the rails only through the thin straps.

```
create_pg_ring_pattern core_ring ...
create_pg_mesh_pattern core_mesh ...
create_pg_mesh_pattern rail_straps ... ;# thin M2 straps that take one via to each rail
create_pg_std_cell_conn_pattern cell_rails ...
compile_pg -strategies $strategies -via_rule pg_vias
check_pg_connectivity -nets {VDD VSS} > $OUT/rpt/pg_conn_pg.rpt
```

At 32 nm, connecting a wide stripe straight down to a 0.06 μm rail never came out clean. One version blocked ten routing tracks and caused 24 shorts. Another gave 306 spacing errors. The kit's own lab scripts use thin straps with one connection to each rail. That design gave 0 errors.

```
user@server: ~/flash/runs/tutorial_ibex_saed32

[user@server tutorial_ibex_saed32]$ grep "^FLASH_PG" fc_setup.log
FLASH_PG_TERMINALS 2
FLASH_PG_FLOATING_WIRES 0 0
FLASH_PG_ROW_BLOCKAGES 0
[user@server tutorial_ibex_saed32]$ grep -E "floating (wires|vias|std cells)" rpt/pg_conn_pg.rpt
Number of floating wires: 0
Number of floating vias: 0
Number of floating std cells: 8477
Number of floating wires: 0
Number of floating vias: 0
Number of floating std cells: 8477
[user@server tutorial_ibex_saed32]$ tail -1 rpt/pg_drc_floorplan.rpt
No errors found.
[user@server tutorial_ibex_saed32]$
```

Figure 11. The power grid checks show no floating wires or vias and no power grid errors.

A floating wire is not connected to the rest of the grid. The same report calls all 8,477 cells floating. That is normal here. No cell touches a rail yet because none is placed. This number only matters after placement, where it must be 0.

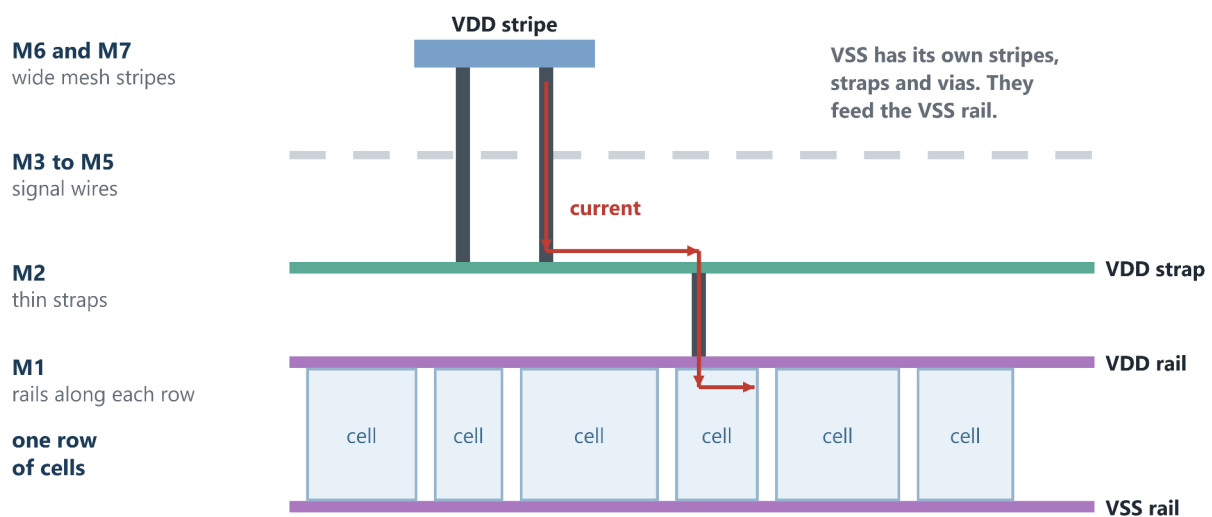


Figure 12. How power reaches a cell, seen from the side. Current comes down from the wide mesh stripes, through vias and a thin strap, to the rail that runs along each row of cells.

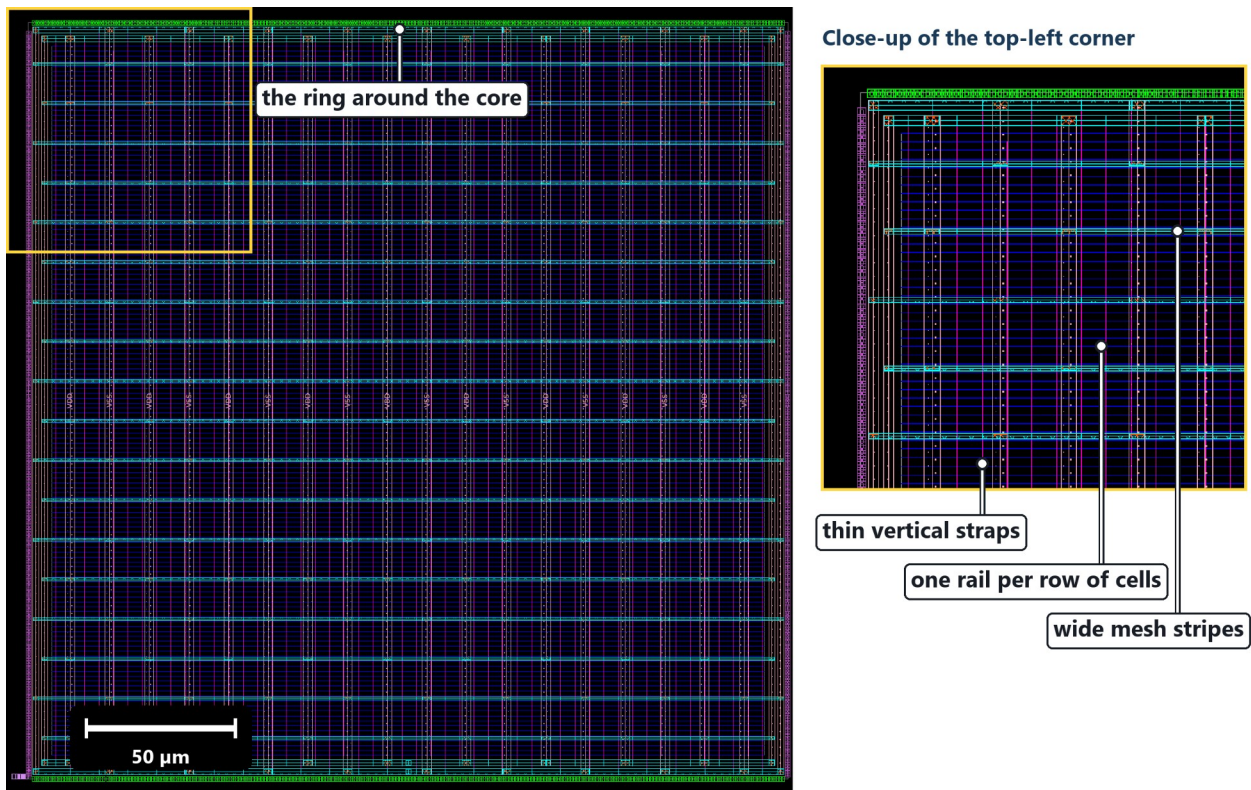


Figure 13. The real power grid from above. The ring circles the core. The close-up shows the wide mesh stripes, the thin vertical straps and the rails, one per row of cells.

The power grid is done when the report shows no floating wires or vias and no power grid errors.

2.3 Placement

First the script puts a buffer on every input pin that drives logic. The clock input gets none. In this run that came to 157 buffers. During manufacturing, a long wire collects electric charge. That charge can damage the transistor the wire connects to. The charge can build up a voltage high enough to break the very thin insulator inside the transistor. This is called an antenna problem. The buffer gives each long input wire a driver. A driver's output can drain that charge before it does harm. Next, one command places all the cells and improves the timing.

```
add_buffer -lib_cell [lib_cells_named [list $DRIVE_CELL]] $in_ports
compile_fusion -from initial_place
legalize_placement
check_legality -verbose > $OUT/rpt/legality_place.rpt
```

```

user@server: ~/flash/runs/tutorial_ibex_saed32

[user@server tutorial_ibex_saed32]$ grep -A9 "^Context" rpt/place_qor.rpt
Context
-----
func_slow      (Setup)      0.004      0.000      0
func_typ       (Setup)      1.108      0.000      0
Design         (Setup)      0.004      0.000      0

func_fast      (Hold)       -0.031     -0.312     53
func_typ       (Hold)       0.033      0.000      0
Design         (Hold)       -0.031     -0.312     53
-----

[user@server tutorial_ibex_saed32]$ grep "check_legality succeeded" rpt/legality_place.rpt
check_legality succeeded.
[user@server tutorial_ibex_saed32]$ grep "floating std cells" rpt/pg_conn_place.rpt
Number of floating std cells: 0
Number of floating std cells: 0
[user@server tutorial_ibex_saed32]$

```

Figure 14. The results after placement show timing at each corner and the legality check. They also show that every cell has power.

Setup now passes at the slow corner with +0.004 ns to spare. The reset problem from stage 1 is gone. Hold fails at the fast corner by 0.031 ns at 53 endpoints. That is expected. Until the clock tree exists, the tool pretends the clock reaches every flip-flop at the same instant. Fusion Compiler can fix hold only after it builds the real tree.

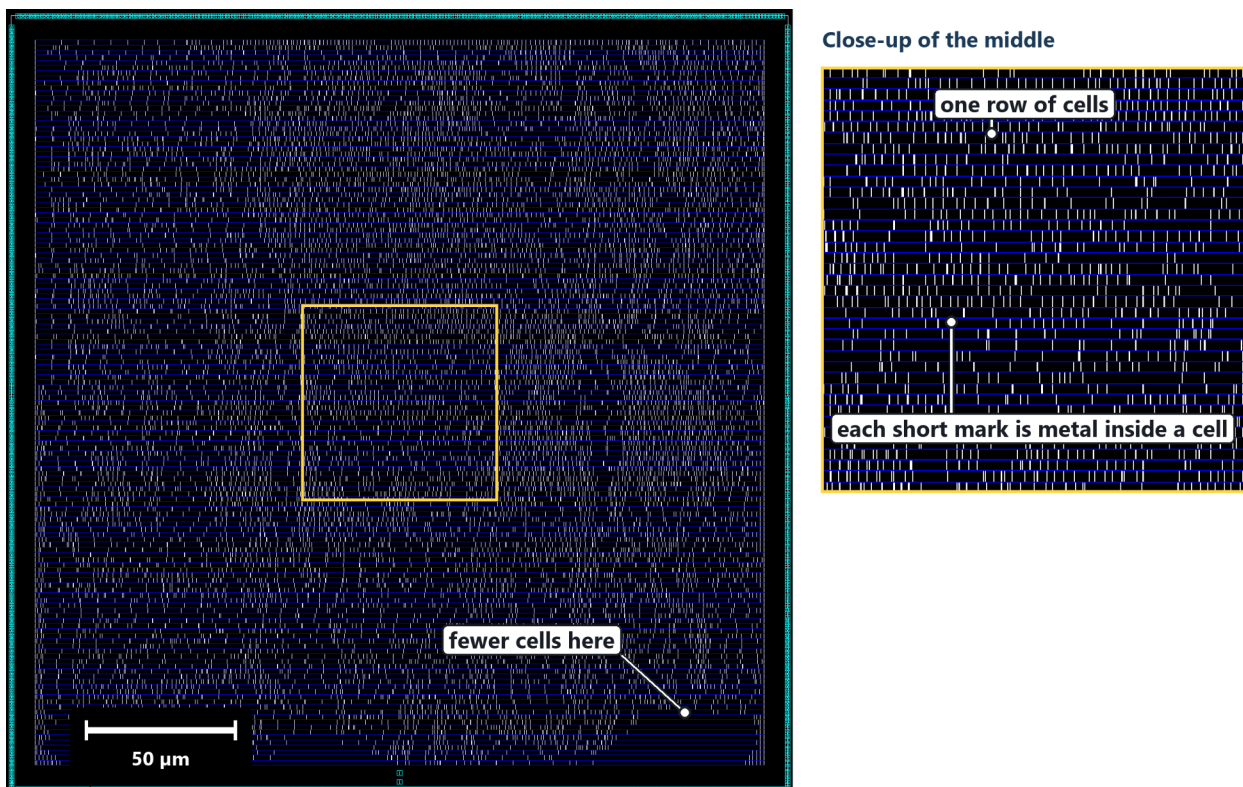


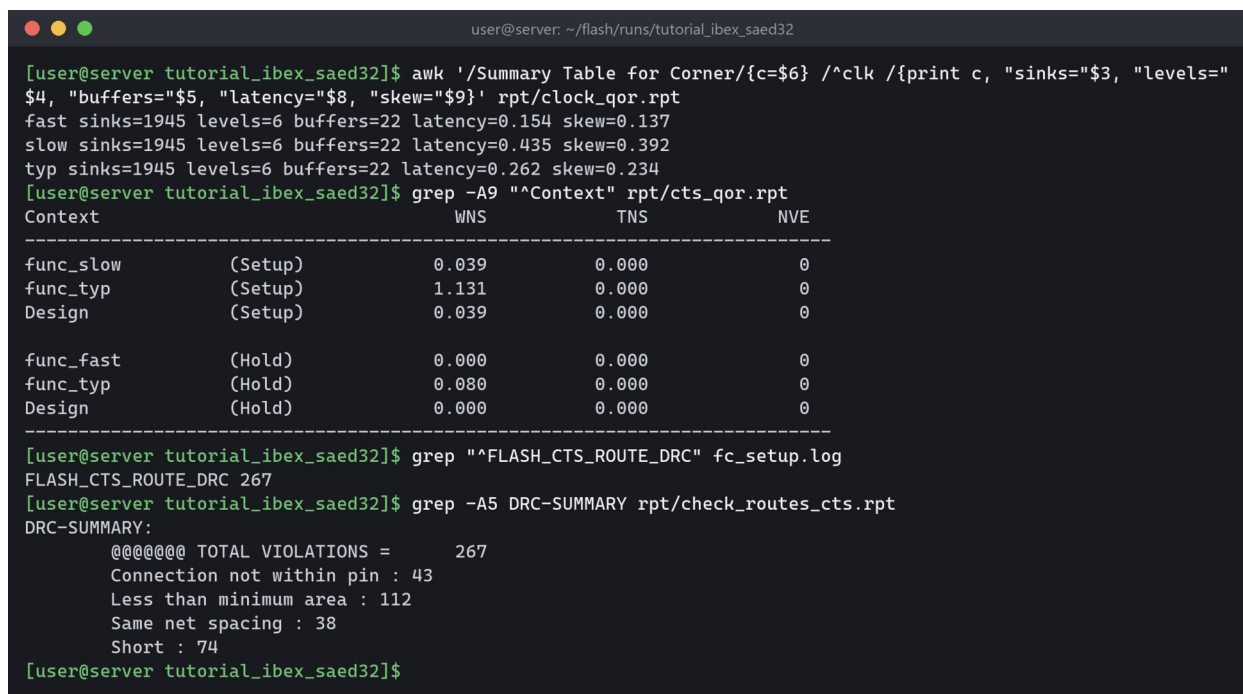
Figure 15. Placement, with only metal layer M1 shown and the picture brightened. The close-up shows the rows. Each short mark is metal inside a placed cell. The lower right has fewer cells.

In a legal placement, every cell sits on a row and no two cells overlap. Placement is done when check_legality passes and every cell has power. Setup should also be close to passing.

2.4 Clock tree

Every flip-flop needs the clock at nearly the same moment. One wire cannot drive them all fast enough. Reset had the same problem in stage 1. clock_opt builds a tree of buffers that carries the clock to every flip-flop. It also draws the clock wires and fixes setup and hold with the real clock. The script adds decap cells now too. A decap is a small capacitor between VDD and VSS. When many cells switch at once, the decaps give up some of their stored charge, so the supply dips less. They have their own M1 metal. That is why they go in before the signal wires. The router works around them.

```
clock_opt
create_stdcell_fillers -lib_cells [lib_cells_named $DECAP_CELLS]
report_clock_qor > $OUT/rpt/clock_qor.rpt
check_routes -open_net false > $OUT/rpt/check_routes_cts.rpt
```



```
[user@server tutorial_ibex_saed32]$ awk '/Summary Table for Corner/{c=$6} /^clk/{print c, "sinks="$3, "levels="$4, "buffers="$5, "latency="$8, "skew="$9}' rpt/clock_qor.rpt
fast sinks=1945 levels=6 buffers=22 latency=0.154 skew=0.137
slow sinks=1945 levels=6 buffers=22 latency=0.435 skew=0.392
typ sinks=1945 levels=6 buffers=22 latency=0.262 skew=0.234
[user@server tutorial_ibex_saed32]$ grep -A9 "^Context" rpt/cts_qor.rpt
Context
-----
func_slow      (Setup)      0.039      0.000      0
func_typ       (Setup)      1.131      0.000      0
Design         (Setup)      0.039      0.000      0
-----
func_fast      (Hold)       0.000      0.000      0
func_typ       (Hold)       0.080      0.000      0
Design         (Hold)       0.000      0.000      0
-----
[user@server tutorial_ibex_saed32]$ grep "^FLASH_CTS_ROUTE_DRC" fc_setup.log
FLASH_CTS_ROUTE_DRC 267
[user@server tutorial_ibex_saed32]$ grep -A5 DRC-SUMMARY rpt/check_routes_cts.rpt
DRC-SUMMARY:
    @@@@ TOTAL VIOLATIONS =      267
    Connection not within pin : 43
    Less than minimum area : 112
    Same net spacing : 38
    Short : 74
[user@server tutorial_ibex_saed32]$
```

Figure 16. Clock tree results. At every corner the tree reaches 1,945 flip-flops with 22 buffers in 6 levels. Hold at fast shows 0.000 with 0 failing endpoints. The last two commands show the 267 errors on the clock wires.

The clock tree reaches 1,945 flip-flops through 22 buffers in 6 levels. Synthesis made 2,014 cells that work with the clock, and the other 69 are clock gates. A clock gate saves power by turning the clock off for a group of flip-flops that has nothing to do.

At the slow corner the clock takes up to 0.435 ns to arrive. This delay is called latency. The earliest and latest arrivals differ by 0.392 ns. This difference is called skew. Skew takes time away from some paths, so smaller is better. Here it is about 8% of the 5 ns tick, and timing still passes. Hold now passes at the fast corner. Setup has +0.039 ns to spare.

The check on the clock wires found 267 errors, all on M3 and below. A clean clock tree should have 0 errors here. Our flow does not meet that yet. It prints the count as FLASH_CTS_ROUTE_DRC in `fc_setup.log` and keeps going. In this run, routing fixed all 267 later. Write the number down and check that routing clears it.

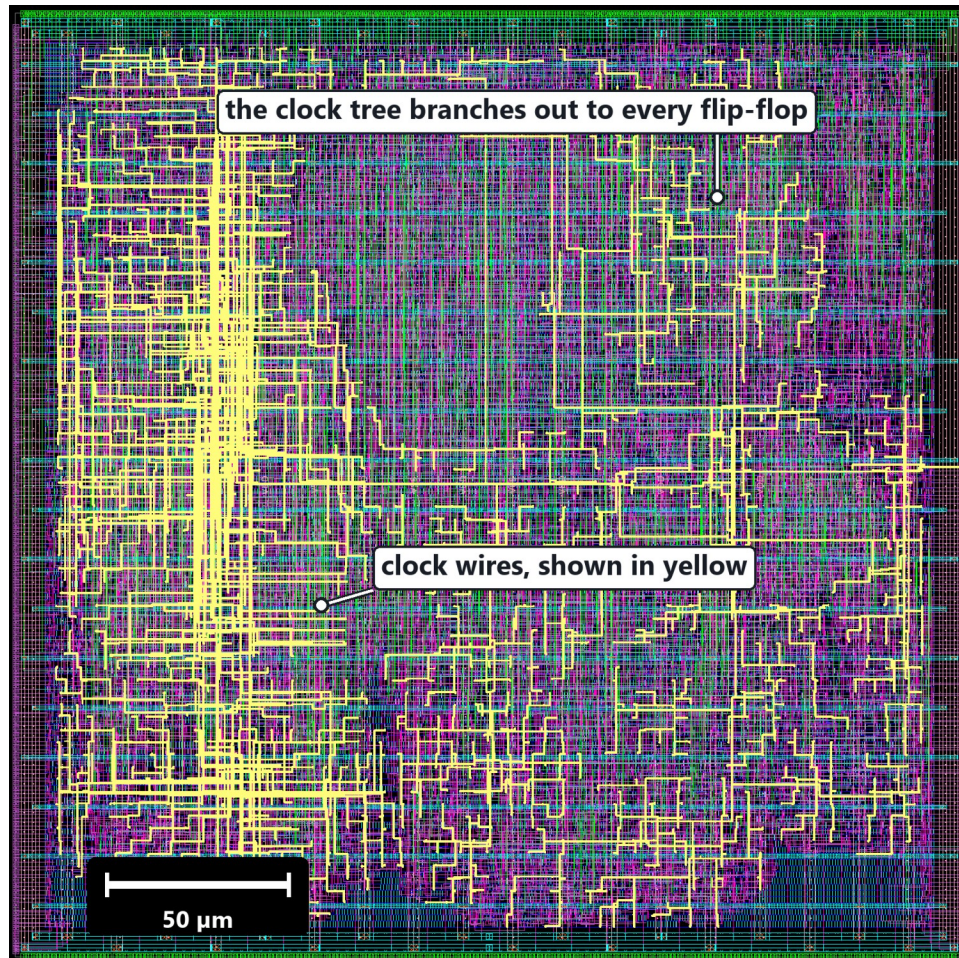


Figure 17. The clock tree, in yellow. Its branches reach every flip-flop in the block.

The clock tree is done when the clock reaches every flip-flop and hold passes at the fast corner. You should also have written down the clock wire error count.

2.5 Routing

Routing turns every net in the netlist into real metal wires and vias. `route_auto` draws every wire. `route_opt` then fixes timing with the real wires. That can break a clean route. The script therefore checks the wires again and runs up to 3 cleanup passes. The router does not know every factory rule. As a last step, `icv_fix_drc` runs IC Validator inside Fusion Compiler. The router then fixes what IC Validator finds. This step is called the in-design DRC.

```
route_auto
route_opt
# up to 3 cleanup passes remove and redraw the wires that have errors
check_routes > $OUT/rpt/route_cleanup_$pass.rpt
icv_fix_drc      ;# the in-design DRC runs IC Validator in Fusion Compiler
```

```

user@server: ~/flash/runs/tutorial_ibex_saed32

[user@server tutorial_ibex_saed32]$ grep -E "^FLASH_(ROUTE_CLEANUP|ICV_FIX)" fc_setup.log
FLASH_ROUTE_CLEANUP pass=1 drcs=0 signal_nets=0
FLASH_ICV_FIX targeted=0 remaining=0
[user@server tutorial_ibex_saed32]$ grep -A9 "^Context" rpt/route_qor.rpt
Context
-----
func_slow      (Setup)      0.001      0.000      0
func_typ       (Setup)      1.119      0.000      0
Design         (Setup)      0.001      0.000      0

func_fast      (Hold)       -0.001     -0.001      4
func_typ       (Hold)       0.083      0.000      0
Design         (Hold)       -0.001     -0.001      4
-----
[user@server tutorial_ibex_saed32]$

```

Figure 18. The routing cleanup pass and the timing after routing.

The first cleanup pass found 0 errors and redrew nothing. The in-design DRC loaded 265 rules and left 137 of them out on purpose. It found 0 problems. Fusion Compiler's timing after routing shows +0.001 ns setup at slow and -0.001 ns hold at fast at 4 endpoints. These use the tighter 4.85 ns clock. PrimeTime gives the real answer in stage 3.

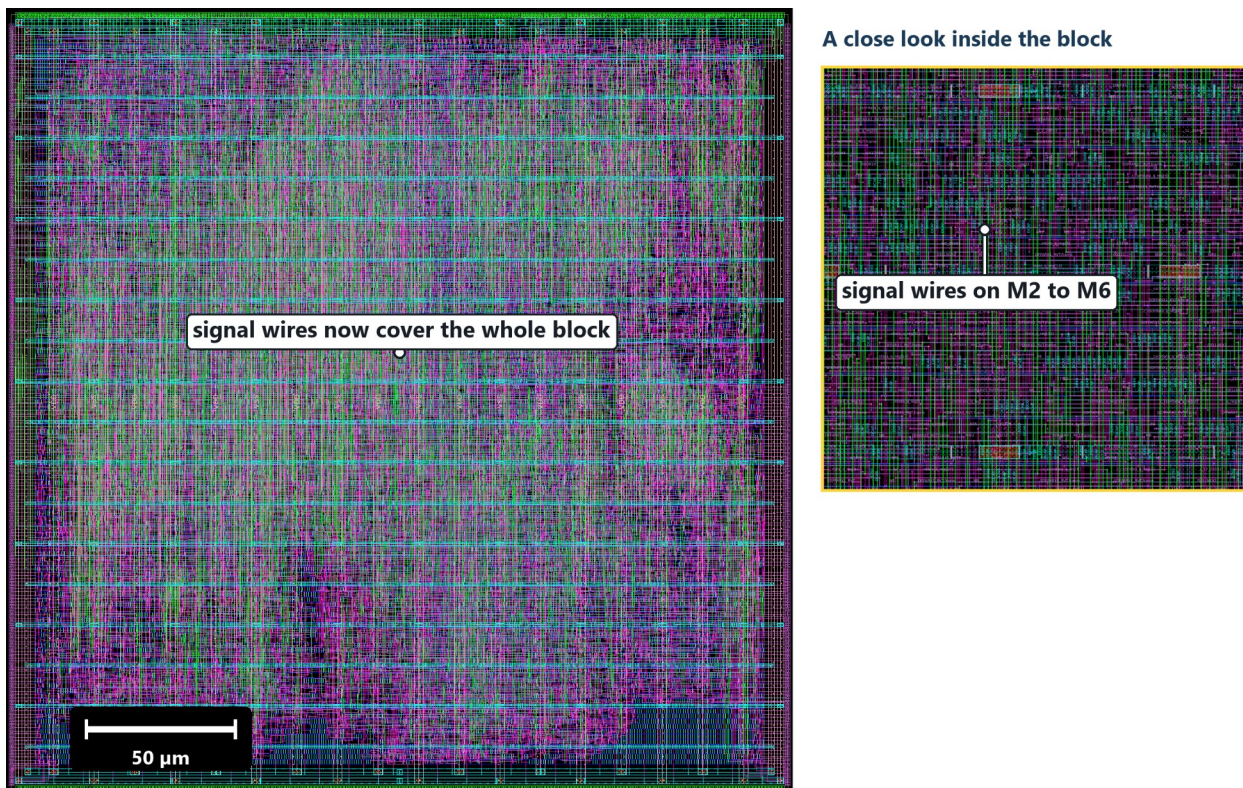


Figure 19. The routed block. Signal wires now cover it. The close-up on the right shows the signal wires on M2 to M6.

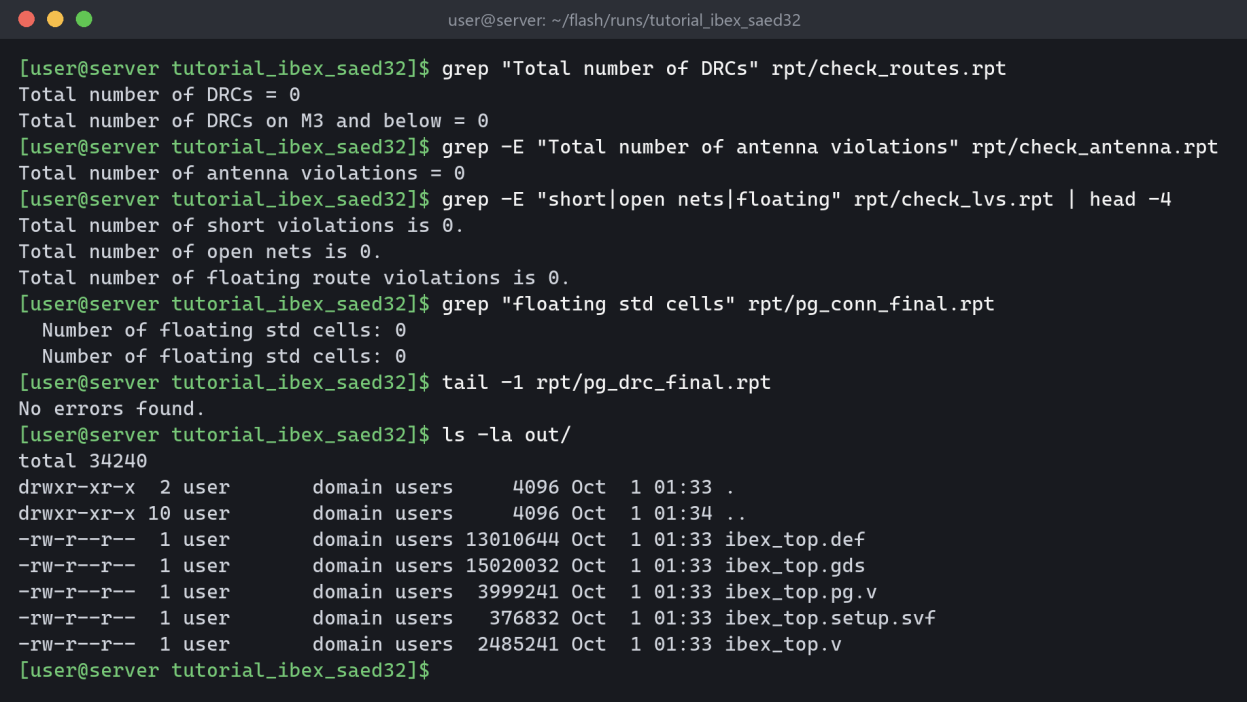
Removing and redrawing wires fixes small, local problems. If more than 300 wires still have errors, the block is out of room. The cleanup then gives up and the run goes on with those errors. The log shows FLASH_ROUTE_INFEASIBLE. In that case, lower UTIL or raise MAX_LAYER. In earlier test runs of this design with less room, lowering UTIL from 0.60 to 0.55 removed about 1,500 routing errors. Allowing one more metal layer did the same.

Routing is done when check_routes reports 0 errors and no open wires or shorts.

2.6 Finish and write the files

The last checkpoint fills every gap in the rows with filler cells. Filler cells hold no logic. They keep every row continuous as the factory rules require. After that the script runs the final checks. It writes the files the checking tools need.

```
create_stdcell_fillers -lib_cells [lib_cells_named $FILL_CELLS]
check_routes           > $OUT/rpt/check_routes.rpt
check_lvs -max_errors 0 > $OUT/rpt/check_lvs.rpt
check_pg_drc           > $OUT/rpt/pg_drc_final.rpt
write_verilog ... $OUT/out/$DESIGN.v           ;# for PrimeTime and Formality
write_def   $OUT/out/$DESIGN.def               ;# for StarRC
write_gds    ... $OUT/out/$DESIGN.gds           ;# the layout
```



```
user@server: ~/flash/runs/tutorial_ibex_saed32

[user@server tutorial_ibex_saed32]$ grep "Total number of DRCs" rpt/check_routes.rpt
Total number of DRCs = 0
Total number of DRCs on M3 and below = 0
[user@server tutorial_ibex_saed32]$ grep -E "Total number of antenna violations" rpt/check_antenna.rpt
Total number of antenna violations = 0
[user@server tutorial_ibex_saed32]$ grep -E "short|open nets|floating" rpt/check_lvs.rpt | head -4
Total number of short violations is 0.
Total number of open nets is 0.
Total number of floating route violations is 0.
[user@server tutorial_ibex_saed32]$ grep "floating std cells" rpt/pg_conn_final.rpt
Number of floating std cells: 0
Number of floating std cells: 0
[user@server tutorial_ibex_saed32]$ tail -1 rpt/pg_drc_final.rpt
No errors found.
[user@server tutorial_ibex_saed32]$ ls -la out/
total 34240
drwxr-xr-x  2 user      domain users    4096 Oct  1 01:33 .
drwxr-xr-x 10 user      domain users    4096 Oct  1 01:34 ..
-rw-r--r--  1 user      domain users 13010644 Oct  1 01:33 ibex_top.def
-rw-r--r--  1 user      domain users 15020032 Oct  1 01:33 ibex_top.gds
-rw-r--r--  1 user      domain users  3999241 Oct  1 01:33 ibex_top.pg.v
-rw-r--r--  1 user      domain users  376832 Oct  1 01:33 ibex_top.setup.svf
-rw-r--r--  1 user      domain users  2485241 Oct  1 01:33 ibex_top.v
[user@server tutorial_ibex_saed32]$
```

Figure 20. The last Fusion Compiler checks and the files it wrote.

Every Fusion Compiler check is clean. check_routes reports 0 errors and no shorts or open wires. There are also 0 antenna errors and no power grid errors. Every cell has power. The layout file is 15 MB. From above, the finished block looks like the routed block in section 2.5. The fillers are hidden under the wires.

2.7 Timing and area at each step

PPA means power, performance and area. Miguel's tutorial has a table called "When should I check PPA". Ours is the same table with our numbers from the `rpt/<checkpoint>_qor.rpt` reports. PrimePower estimates power only once, in stage 3. All slacks here use the tighter 4.85 ns clock. In the hold column, the second number is how many endpoints fail. In the last column, congestion means too many wires want the same space. We read the Final row before the fix loop. The fix loop later rewrote `rpt/final_qor.rpt`. It now shows +0.078 ns setup and -0.003 ns hold at 4 endpoints. Its cell area is 39,626 μm^2 .

Step	Setup slack, slow	Hold slack, fast	Cell area, μm^2	What to look at
Synthesis	-4,188.402	-0.033, 67 endpoints	34,634	area and cell count. Reset has no buffer tree yet
Floorplan and power grid	no change	-0.039, 210 endpoints	34,634	block size, pins and power connections
Placement	+0.004	-0.031, 53 endpoints	37,103	legality, setup and congestion
Clock tree	+0.039	0.000, 0 endpoints	39,501	latency, skew and hold
Routing	+0.001	-0.001, 4 endpoints	39,517	route DRC, opens and shorts
Final	+0.001	-0.001, 4 endpoints	39,517	all checks, then signoff

Stage 2 is done when routing has no errors, shorts or opens. Every cell must have power. There must be no antenna errors. Timing should be close to passing. Passing Fusion Compiler's own checks does not finish the job. It means the design is ready for signoff in stage 3.

Stage 3: Signoff

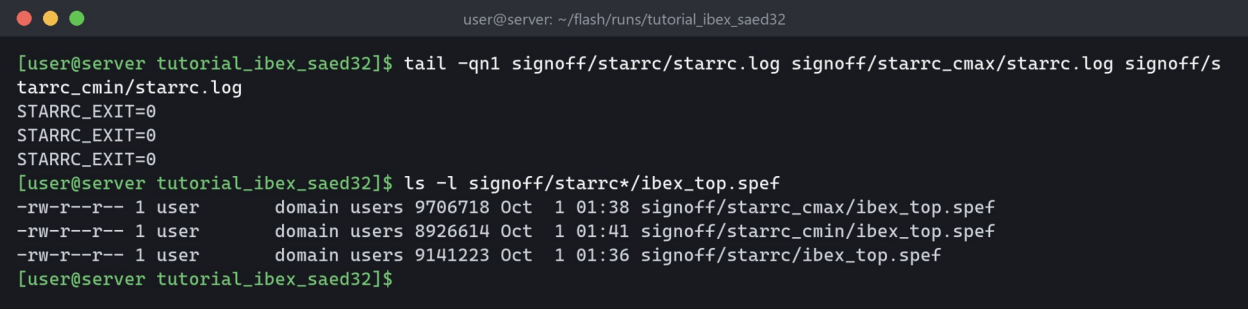
Signoff means checking the design with tools that did not build it. Every serious mistake we found had the same pattern. One tool did something reasonable but wrong. Its own report still looked clean. A second, separate check caught it. That is why signoff is several checks that must all agree. The first command below runs every step. The second runs only the steps you name. Use the names in the Step column of the table.

```
~/flash/flow/signoff.sh ~/flash/runs/tutorial_ibex_saed32
# or run only some steps
~/flash/flow/signoff.sh ~/flash/runs/tutorial_ibex_saed32 pt check
```

Step	Tool	What it checks	Passes when
starrc	StarRC	how much each wire delays its signal	it finishes at all 3 wire corners
pt	PrimeTime SI	timing, crosstalk and electrical limits	every limit is met at all 3 corners
fm	Formality	the netlist does the same logic as the RTL	every point matches
pp	PrimePower	how much power the block uses	it gives a number
drc	IC Validator	the factory's shape rules	0 errors with every rule run
lvs	IC Validator	the layout connects the same circuit as the netlist	the netlists match
check	check.py	all of the above	every check in its list passes and it prints SIGNOFF CLEAN

3.1 Wire delays, StarRC

Every wire has resistance and capacitance. Both slow the signal down. StarRC works out each wire's delay once for each wire corner. The slow corner uses the most delay and the typ corner uses typical delay. The fast corner uses the least delay. If one wire corner is missing, the script stops. It will not use the typical one in its place. StarRC is done when all three wire corners finish.

A terminal window with a dark background and light green text. The prompt is 'user@server: ~/flash/runs/tutorial_ibex_saed32'. The user enters a command to tail three log files. The output shows 'STARRC_EXIT=0' repeated three times. Then the user enters a command to list files with permissions. The output shows three files with permissions '-rw-r--r--', owner 'user', group 'domain', size, date, and filename. The files are 'signoff/starrc_cmax/ibex_top.spef', 'signoff/starrc_cmin/ibex_top.spef', and 'signoff/starrc/ibex_top.spef'. The prompt returns at the end.

```
user@server: ~/flash/runs/tutorial_ibex_saed32
[user@server tutorial_ibex_saed32]$ tail -qn1 signoff/starrc/starrc.log signoff/starrc_cmax/starrc.log signoff/s
tarrc_cmin/starrc.log
STARRC_EXIT=0
STARRC_EXIT=0
STARRC_EXIT=0
[user@server tutorial_ibex_saed32]$ ls -l signoff/starrc*/ibex_top.spef
-rw-r--r-- 1 user      domain users 9706718 Oct  1 01:38 signoff/starrc_cmax/ibex_top.spef
-rw-r--r-- 1 user      domain users 8926614 Oct  1 01:41 signoff/starrc_cmin/ibex_top.spef
-rw-r--r-- 1 user      domain users 9141223 Oct  1 01:36 signoff/starrc/ibex_top.spef
[user@server tutorial_ibex_saed32]$
```

Figure 21. StarRC finished all three wire corners.

3.2 Timing, PrimeTime SI

PrimeTime reads the netlist and the wire delays. At each corner it checks setup and hold timing. It also checks electrical limits and crosstalk. Start by checking that every wire got its delays. Each pt line in

signoff/CHECK.txt must show pin_to_pin_not_annotated=0. That value means no wire is missing its delays. Once, a wrong file path let PrimeTime pass a design with no wire delays at all. The checker now requires every wire to have them.

PrimeTime is stricter than Fusion Compiler on purpose. It adds the 5% delay penalty to cover small differences between transistors on a real chip. It also uses the most-delay wire corner at the slow corner. Expect PrimeTime's slack to be lower than Fusion Compiler's.

```

user@server: ~/flash/runs/tutorial_ibex_saed32

[user@server tutorial_ibex_saed32]$ grep -h "^FLASH_PT corner" signoff/pt_slow/pt.log signoff/pt_typ/pt.log signoff/pt_fast/pt.log
FLASH_PT corner=slow period=5.0 setup_wns=-0.140062 hold_wns=0.233472 setup_wns_flops=-0.140062
FLASH_PT corner=typ period=5.0 setup_wns=0.648484 hold_wns=0.111789 setup_wns_flops=1.150328
FLASH_PT corner=fast period=5.0 setup_wns=0.919527 hold_wns=0.034272 setup_wns_flops=1.879752
[user@server tutorial_ibex_saed32]$ grep -h "^FLASH_PT_NOISE" signoff/pt_slow/pt.log signoff/pt_typ/pt.log signoff/pt_fast/pt.log
FLASH_PT_NOISE corner=slow worst_slack=0.0 violators=0
FLASH_PT_NOISE corner=typ worst_slack=0.4 violators=0
FLASH_PT_NOISE corner=fast worst_slack=0.44 violators=0
[user@server tutorial_ibex_saed32]$ grep -m4 VIOLATED signoff/pt_slow/violators.rpt
gen_regfile_ff.register_file_i/g_plain_rf.g_rf_flops[28].rf_reg_q_reg[31]/D -0.14 (VIOLATED)
gen_regfile_ff.register_file_i/g_plain_rf.g_rf_flops[12].rf_reg_q_reg[31]/D -0.14 (VIOLATED)
gen_regfile_ff.register_file_i/g_plain_rf.g_rf_flops[14].rf_reg_q_reg[31]/D -0.13 (VIOLATED)
gen_regfile_ff.register_file_i/g_plain_rf.g_rf_flops[7].rf_reg_q_reg[31]/D -0.13 (VIOLATED)
[user@server tutorial_ibex_saed32]$

```

Figure 22. PrimeTime at the three corners. Setup slack is negative only at slow, -0.140 ns. Crosstalk noise has 0 violators at every corner. The worst setup paths end at bit 31 of the register file.

Every wire has its delays at every corner. That is 11,107 wires, with none missing. Setup fails at the slow corner by 0.140 ns. The worst paths end at bit 31 of the register file. The register file is the processor's set of 32 small storage slots. Fusion Compiler said +0.001 ns even though it timed the tighter 4.85 ns clock. It had no 5% delay penalty and used typical wire delays. Some cells are also too weak for the wires they drive. They break their electrical limits. There are 66 of these at slow and 42 at typical. The fast corner has 11. The worst are clock buffers driving 70.62 fF when their limit is 64 fF. A cell over its limit makes slow edges. Slow edges make the next cells slow too, and the timing models stop being accurate. Hold and crosstalk pass at every corner.

Corner	Wire corner	Setup slack, ns	Hold slack, ns	Electrical limit failures	Crosstalk failures
slow	most delay	-0.140	+0.233	66	0
typ	typical	+0.648	+0.112	42	0
fast	least delay	+0.920	+0.034	11	0

3.3 Same logic as the code, Formality

Formality proves point by point that the final netlist does the same logic as the RTL. It does not try inputs one at a time, as a simulation does. It turns both into logic equations and proves they match for every possible input. A point is a place Formality compares, such as a flip-flop or an output pin. It reads guide files from Fusion Compiler that list the changes synthesis made. They are the out/ibex_top.*.svf files. Formality checks logic only. It says nothing about timing or about the layout. Those are PrimeTime's and IC Validator's jobs.

```
user@server: ~/flash/runs/tutorial_ibex_saed32

[user@server tutorial_ibex_saed32]$ grep -E "^Verification (SUCCEEDED|FAILED)|compare points" signoff/formality/
fm.log
0(69) Unmatched reference(implementation) compare points
0(69) Unmatched reference(implementation) compare points
Verification SUCCEEDED
2372 Passing compare points
[user@server tutorial_ibex_saed32]$
```

Figure 23. Formality matches all 2,372 points, with none failed and none skipped.

Formality checked all 2,372 points and skipped none. None failed. The 69 clock gates exist only in the netlist. They are not in the RTL, so Formality has nothing to compare them with. That is why a person still has to review them, as the section "End of the block run" explains.

3.4 Power, PrimePower

PrimePower adds up the power at the typical corner. It makes every input pin switch 0.2 times per clock cycle. From the logic, it works out how often each wire inside switches. The result is therefore an estimate. Switching recorded from a real program gives a better estimate for that program. That estimate still comes from the library models.

```
user@server: ~/flash/runs/tutorial_ibex_saed32

[user@server tutorial_ibex_saed32]$ grep -E "Switching Power|Internal Power|Leakage Power|^Total Power" signoff/
primepower/power.rpt
Net Switching Power = 3.088e-04 ( 6.05%)
Cell Internal Power = 5.298e-04 (10.39%)
Cell Leakage Power = 4.262e-03 (83.56%)
Total Power = 5.100e-03 (100.00%)
[user@server tutorial_ibex_saed32]$
```

Figure 24. PrimePower's report is in watts. The total of 5.100e-03 W is 5.10 mW. Leakage is the largest part.

The block uses 5.10 mW in total. Switching wires take 0.31 mW and power inside the cells is 0.53 mW. The other 4.26 mW is leakage, current that trickles through transistors even when they are switched off. In this kit leakage is 84% of the total.

3.5 Factory rules and connections, IC Validator

DRC checks every shape in the layout against the factory's rules. LVS rebuilds the circuit from the layout and compares it with the netlist. Both run on the real layout with the full drawing of every cell.

```
user@server: ~/flash/runs/tutorial_ibex_saed32

[user@server tutorial_ibex_saed32]$ head -1 signoff/drc/ibex_top.RESULTS
RESULTS: CLEAN
[user@server tutorial_ibex_saed32]$ tail -1 signoff/drc.log
DRC_EXIT=0
[user@server tutorial_ibex_saed32]$ head -1 signoff/lvs/run/ibex_top.RESULTS
LVS Compare Results: PASS
[user@server tutorial_ibex_saed32]$ grep -i "final comparison result" signoff/lvs/run/ibex_top.LVS_ERRORS
Final comparison result:PASS
[user@server tutorial_ibex_saed32]$
```

Figure 25. DRC and LVS results. For DRC, look for RESULTS: CLEAN and DRC_EXIT=0. For LVS, look for PASS on both lines.

DRC is CLEAN, with every rule run. LVS passes. The layout connects the same circuit as the netlist.

Before we trust a check, we plant a mistake on purpose. For LVS we swap two wires. For DRC we draw a shape that is too thin. Twice, a clean result came from a check that was not running. The planted mistake caught it both times.

3.6 The final answer, check.py

check.py reads every report and writes one line for each check to signoff/CHECK.txt. A check fails if its report is missing. A step that never ran therefore cannot pass. It also fails if its inputs or reports changed after check.py recorded their fingerprints. The last line is the answer.

```

user@server: ~/flash/runs/tutorial_ibex_saed32

[user@server tutorial_ibex_saed32]$ grep -E "(fc_|starrc |pt_(slow|typ|fast) |noise_|formality |primepower |drc |lvs )" signoff/CHECK.txt | cut -c1-105
fc_flow PASS done=True errors=0
fc_route_drc PASS Total number of DRCs = 0
fc_antenna PASS violations=0 nets skipped (ZRT-311)=0
fc_check_lvs PASS short violations=0; open nets=0; floating route violations=0
fc_pg_drc PASS No errors found.
fc_pg_connectivity PASS floating counts 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
starrc PASS exit=0 errors=0
pt_fast PASS period=5.0ns setup_wns=0.920 hold_wns=0.034 spef_nets=11107 pi
noise_fast PASS worst_slack=0.44 violators=0
pt_slow FAIL period=5.0ns setup_wns=-0.140 hold_wns=0.233 spef_nets=11107 p
noise_slow PASS worst_slack=0.0 violators=0
pt_typ PASS period=5.0ns setup_wns=0.648 hold_wns=0.112 spef_nets=11107 pi
noise_typ PASS worst_slack=0.4 violators=0
formality PASS SUCCEEDED Passing=2372
primepower PASS mode=vectorless_toggle0.2 total=0.005100152173510245 W
drc PASS icv violations=0 not_executed=0
lvs PASS netlists match
[user@server tutorial_ibex_saed32]$ grep FAIL signoff/CHECK.txt | cut -c1-105
pt_slow FAIL period=5.0ns setup_wns=-0.140 hold_wns=0.233 spef_nets=11107 p
drc_waiver_review FAIL missing or malformed reviewed global DRC waiver record
electrical_slow FAIL unwaived=66 waived=0 first=ZCTSBUF_268_25122/Y 64.00
timing_constraints_slow FAIL violations=105 first=gen_regfile_ff.register_file_i/g_plain_rf
pt_report_analysis_coverage_slow FAIL missing reviewed timing coverage contract
electrical_typ FAIL unwaived=42 waived=0 first=ZCTSBUF_268_25122/Y 64.00
pt_report_analysis_coverage_typ FAIL missing reviewed timing coverage contract
electrical_fast FAIL unwaived=11 waived=0 first=ZCTSBUF_268_25122/Y 64.00
pt_report_analysis_coverage_fast FAIL missing reviewed timing coverage contract
formality_coverage FAIL passing=2372 excluded=0 clock_gate_latches=1; requires reviewe
[user@server tutorial_ibex_saed32]$ tail -1 signoff/CHECK.txt
SIGNOFF NOT CLEAN: 79 of 89 checks pass
[user@server tutorial_ibex_saed32]$

```

Figure 26. The checker's answer for this run. Each line is one check, with PASS or FAIL and the numbers behind it. The last line says 79 of 89 checks pass.

79 of 89 checks pass. The 10 failures are of two kinds. Five are real problems in the design. They are setup at the slow corner, the report that lists the failing setup paths and the electrical limits at the three corners. The fix loop in the next section fixes them. The other five fail because a review record is missing. A review record is where a person reads the evidence and signs off on something the tools cannot judge. The section "End of the block run" explains them.

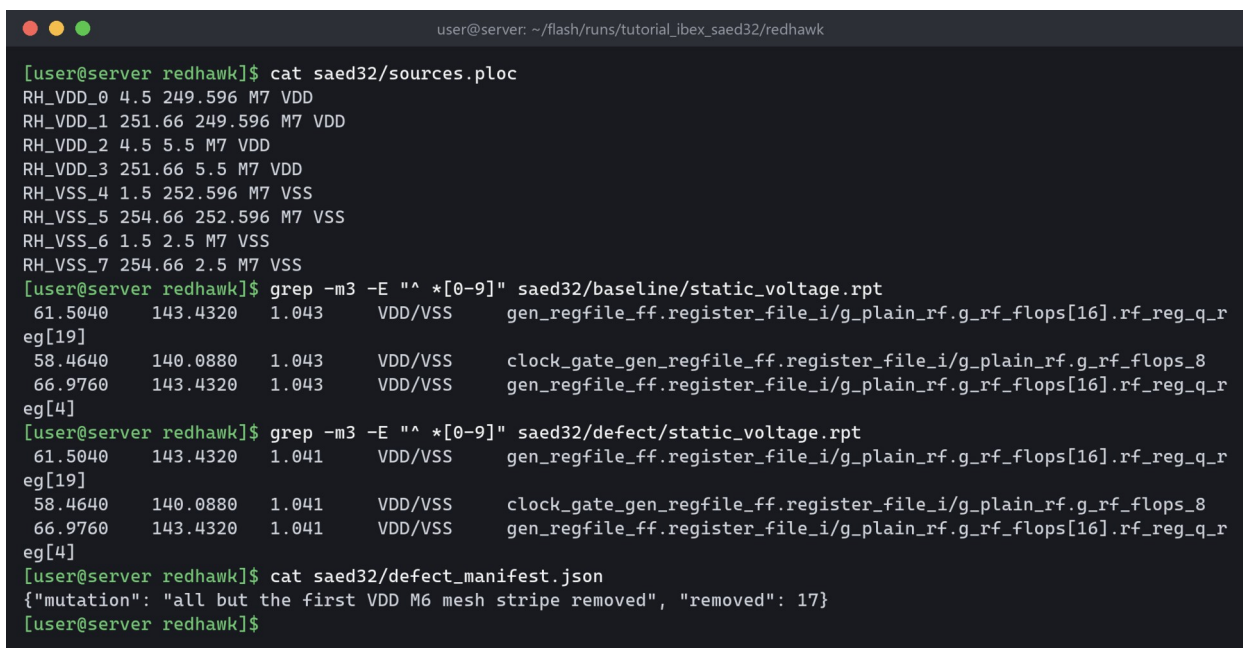
3.7 Supply voltage drop, RedHawk-SC

Every cell draws current from the power grid. The grid's metal has resistance. That makes the supply voltage sag a little on its way to each cell. This sag is called IR drop. Too much of it slows cells down.

RedHawk-SC measures it. It is not part of `signoff.sh`. We ran it on its own with the commands below. For the chip, `chip/ir/ir.sh` does all of this, as section 6.6 shows.

```
RUN=~/.flash/runs/tutorial_ibex_saed32
python3 rh_setup.py $RUN # copies the layout and finds the ring corners
cd $RUN/redhawk
python3 run.py saed32 baseline # the real power grid
python3 run.py saed32 defect # planted control with a weaker grid
```

The setup puts four ideal power sources on each supply at the corners of the power ring. There is no model of the package or the bond wires, so the real drop would be somewhat larger. RedHawk works out its own power from 10% switching activity. It came to about 9.8 mW, more than PrimePower's 5.10 mW. This check assumes more current than PrimePower estimated. That puts the error on the safe side.



```
[user@server redhawk]$ cat saed32/sources.ploc
RH_VDD_0 4.5 249.596 M7 VDD
RH_VDD_1 251.66 249.596 M7 VDD
RH_VDD_2 4.5 5.5 M7 VDD
RH_VDD_3 251.66 5.5 M7 VDD
RH_VSS_4 1.5 252.596 M7 VSS
RH_VSS_5 254.66 252.596 M7 VSS
RH_VSS_6 1.5 2.5 M7 VSS
RH_VSS_7 254.66 2.5 M7 VSS
[user@server redhawk]$ grep -m3 -E "^[0-9]" saed32/baseline/static_voltage.rpt
61.5040 143.4320 1.043 VDD/VSS gen_regfile_ff.register_file_i/g_plain_rf.g_rf_flops[16].rf_reg_q_r
eg[19]
58.4640 140.0880 1.043 VDD/VSS clock_gate_gen_regfile_ff.register_file_i/g_plain_rf.g_rf_flops_8
66.9760 143.4320 1.043 VDD/VSS gen_regfile_ff.register_file_i/g_plain_rf.g_rf_flops[16].rf_reg_q_r
eg[4]
[user@server redhawk]$ grep -m3 -E "^[0-9]" saed32/defect/static_voltage.rpt
61.5040 143.4320 1.041 VDD/VSS gen_regfile_ff.register_file_i/g_plain_rf.g_rf_flops[16].rf_reg_q_r
eg[19]
58.4640 140.0880 1.041 VDD/VSS clock_gate_gen_regfile_ff.register_file_i/g_plain_rf.g_rf_flops_8
66.9760 143.4320 1.041 VDD/VSS gen_regfile_ff.register_file_i/g_plain_rf.g_rf_flops[16].rf_reg_q_r
eg[4]
[user@server redhawk]$ cat saed32/defect_manifest.json
{"mutation": "all but the first VDD M6 mesh stripe removed", "removed": 17}
[user@server redhawk]$
```

Figure 27. RedHawk-SC results. The worst cells see 1.043 V of the 1.05 V supply. With 17 VDD mesh stripes removed, the same cells fall to 1.041 V.

The worst drop is about 7 mV, under 1% of the supply. The defect run is a planted control. It removes 17 VDD mesh stripes on purpose. The worst cells then fall to 1.041 V. That shows the check notices a weaker grid. It also shows the rest of the grid still carries the current. SAED32 gives no limit for voltage drop. This result is for information only. It is far inside the 10% allowance used in section 6.6.

Electromigration is slow wear of a wire from too much current over years. Checking it needs a current limit for each metal layer from the factory. SAED32's technology files have no EM limits. Nobody has turned its rule documents into a form RedHawk can check. For that reason this tutorial claims no EM result. The IHP chip work uses the limits IHP publishes in its process specification.

The fix loop

Everything from here up to stage 4 is still part of stage 3. Signoff is a loop, as in Miguel's tutorial. Find the failing check in CHECK.txt and open its report. Find the exact cell, wire or rule that fails. The table shows where to fix each kind of problem:

Problem	Where to fix it	How
setup, hold or electrical limits at one corner	an ECO that makes a few cells stronger or adds buffers	eco_fixed.sh
routing errors after route_opt	remove and redraw those wires	already in fc_flow.tcl
hundreds of routing errors	the floorplan. Lower UTIL or raise MAX_LAYER	a new run
factory rule errors the router missed	the in-design DRC or a flow change	icv_fix_drc
LVS or Formality mismatch	stop and find the cause. Never explain it away	compare the layout, the netlist and the RTL until you find the difference

ECO rounds

ECO is short for engineering change order. An ECO is a small fix after routing. Our script asks PrimeTime to fix one kind of problem at one corner. PrimeTime writes the changes. For example, it might make ten cells stronger. Fusion Compiler makes those changes and moves only the changed cells. It redraws only their wires and writes new files. The script then runs every check again. If any check that passed before now fails, the script puts the run back as it was.

```
RUN=~/.flash/runs/tutorial_ibex_saed32
~/flash/flow/eco_fixed.sh $RUN slow setup      # fix setup at the slow corner
~/flash/flow/eco_fixed.sh $RUN slow drc        # fix electrical limits
# repeat the drc line until FLASH_ECO_NONE says nothing more can be fixed
```

In these commands, drc means the electrical limits from section 3.2. It is not the factory shape check that IC Validator runs.

This run showed that our first fix script, eco.sh, had four bugs. It checked timing without the 5% delay penalty. It saw +0.071 ns where signoff saw -0.140 ns, so it fixed nothing. It asked PrimeTime for two kinds of fix in one command. PrimeTime 2026.03 refuses that. It did not let the clock buffers it changed move, so they overlapped their neighbors and caused 82 shorts. Last, a failed attempt left a log behind that broke every later check. eco_fixed.sh fixes all four. It is the one in the repository, flow/eco_fixed.sh.

One round, step by step. Signoff found the worst setup path at -0.140 ns. It ends at bit 31 of the register file. The setup round asked PrimeTime to fix setup at the slow corner. PrimeTime picked cells on the failing paths and swapped each for a stronger version of the same cell. Each swap makes one step of a path a little faster. Round 2 swapped 49 cells this way. Fusion Compiler applied the swaps and the script signed off again. Slow setup went from -0.140 ns to +0.001 ns, and no other check broke. That is one kept round.

The table lists the rounds on this run in order. They ran on earlier test versions of `eco_fixed.sh`, and their logs are in the run's `eco_fix/` folder. With the final script your rounds may differ, but the end result should look like the table below the figure. After round 4, PrimeTime could fix no more by making cells stronger. We switched to adding buffers. Rounds 5 to 7 were undone because of script bugs. Round 8 added 5 buffers. That made room for one more stronger cell in round 9.

Round	Kind	What changed	Result
1	electrical	57 cells made stronger, 11 of them clock buffers	Undone. The script did not let the clock buffers move, so they overlapped their neighbors. That caused 82 shorts and 668 DRC errors. LVS failed too.
2	setup	49 cells made stronger	Kept. Slow setup went from -0.140 to +0.001 ns.
3	electrical	56 cells made stronger, 11 of them clock buffers. All could move.	Kept. Slow went from 65 to 10 electrical limit failures. Typ went from 40 to 6 and fast from 11 to 2.
4	electrical	nothing	Making cells stronger could fix no more.
5 to 7	electrical	5 buffers added	Undone each time because of script problems. The design was not the cause. <code>eco_fixed.sh</code> fixes those problems.
8	electrical	5 buffers added	Kept. Electrical limit failures were 5 at slow, 3 at typ and 2 at fast.
9	electrical	1 cell made stronger	Kept. Electrical limit failures were 4 at slow, 3 at typ and 2 at fast. PrimeTime could fix no more.
10	by hand	5 cells picked by hand and made stronger	Kept. No electrical limit failures remain at any corner. 84 of 89 checks pass.

```

user@server: ~/flash/runs/tutorial_ibex_saed32

[user@server tutorial_ibex_saed32]$ grep -h "FLASH_ECO_REJECTED" eco_fix/chain.log
FLASH_ECO_REJECTED 73 checks pass against 79 before; restored the run, the attempt is in ~/flash/runs/tutorial_i
bex_saed32/eco_rejected_eco_pt_slow_drc
[user@server tutorial_ibex_saed32]$ grep -m4 -E "(fc_route_drc|fc_check_lvs|drc|lvs)" eco_fix/chain.log | sed
"s/(violations=[0-9]*\).*\/\1/"
fc_route_drc          FAIL      Total number of DRCs = 1
fc_check_lvs          FAIL      short violations=82
drc                   FAIL      icv violations=668
lvs                   FAIL      Generating ibex_top.LAYOUT_ERRORS...
[user@server tutorial_ibex_saed32]$ awk '/^FLASH_PT_ECO/{print FILENAME": "$3, $5, $6, $NF}' eco_fix/round2_eco_
pt_slow_setup/pt.log eco_fix/round3_eco_pt_slow_drc/pt.log eco_fix/round8_eco_pt_slow_drc/pt.log eco_fix/round9_
eco_pt_slow_drc/pt.log eco_fix/round10_eco_pt_slow_manual/pt.log
eco_fix/round2_eco_pt_slow_setup/pt.log: type=setup setup_wns_before=-0.140062 setup_wns_after=0.000714 changes=
56
eco_fix/round3_eco_pt_slow_drc/pt.log: type=drc setup_wns_before=0.001078 setup_wns_after=0.006995 changes=63
eco_fix/round8_eco_pt_slow_drc/pt.log: type=drc setup_wns_before=0.006953 setup_wns_after=0.006953 changes=12
eco_fix/round9_eco_pt_slow_drc/pt.log: type=drc setup_wns_before=0.006957 setup_wns_after=0.006957 changes=8
eco_fix/round10_eco_pt_slow_manual/pt.log: type=manual setup_wns_before=0.006957 setup_wns_after=0.006583 change
s=12
[user@server tutorial_ibex_saed32]$ grep -h "FLASH_ECO_(RELEASED|LEGALITY_PASS)" fc_final.log
FLASH_ECO_RELEASED 5 of 5
FLASH_ECO_LEGALITY_PASS
[user@server tutorial_ibex_saed32]$

```

Figure 28. Each round's PrimeTime summary and the placement check after the last round. The `changes=` field counts every line PrimeTime wrote. It does not count changed cells.

Round 2 fixed setup. It also cut the electrical limit failures at slow from 66 to 65 and at typ from 42 to 40. Fast stayed at 11. Together the kept rounds made 111 cell changes and added 5 buffers. Setup

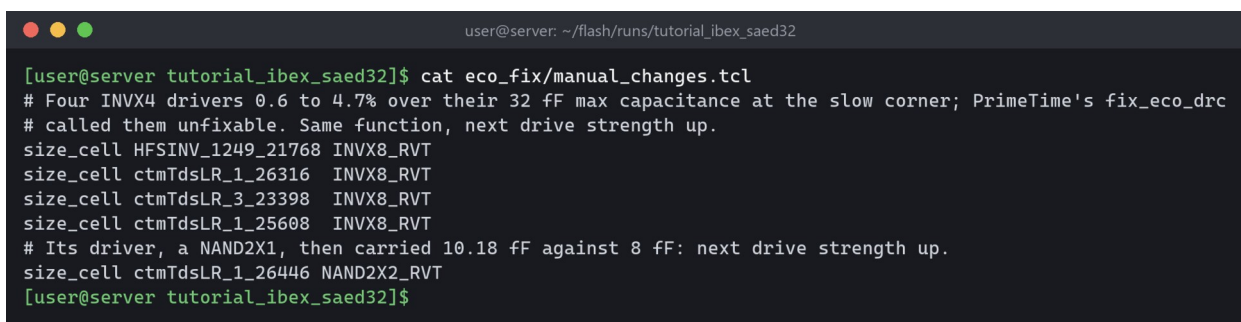
passes at every corner. The slow corner has +0.006 ns to spare. Every electrical limit is met. Hold and crosstalk stayed clean in every round we kept. Formality, DRC and LVS did too.

Corner	Setup before	Setup after	Hold after	Electrical failures before	Electrical failures after
slow	-0.140	+0.006	+0.218	66	0
typ	+0.648	+0.649	+0.105	42	0
fast	+0.920	+0.920	+0.030	11	0

The last few by hand

After round 9, four cells at the slow corner were still 0.6% to 4.7% over a 32 fF limit. PrimeTime said it could not fix them. All four were INVX4 inverters. The usual fix is the next size up, INVX8. A bigger cell also puts a heavier load on the cell that drives it. We first tried the change at all three corners in PrimeTime alone. The design stayed untouched.

The first try left one new problem. The NAND2X1 driving one of the inverters went to 10.18 fF, over its 8 fF limit. Making that one a NAND2X2 as well cleared everything. Slow setup stayed positive. We then applied the list with `eco_fixed.sh`, which runs every check again.



```

[user@server tutorial_ibex_saed32]$ cat eco_fix/manual_changes.tcl
# Four INVX4 drivers 0.6 to 4.7% over their 32 fF max capacitance at the slow corner; PrimeTime's fix_eco_drc
# called them unfixable. Same function, next drive strength up.
size_cell HFSINV_1249_21768 INVX8_RVT
size_cell ctmTdsLR_1_26316 INVX8_RVT
size_cell ctmTdsLR_3_23398 INVX8_RVT
size_cell ctmTdsLR_1_25608 INVX8_RVT
# Its driver, a NAND2X1, then carried 10.18 fF against 8 fF: next drive strength up.
size_cell ctmTdsLR_1_26446 NAND2X2_RVT
[user@server tutorial_ibex_saed32]$

```

Figure 29. The hand-written change list. Each line keeps the same logic and steps up one size.

```

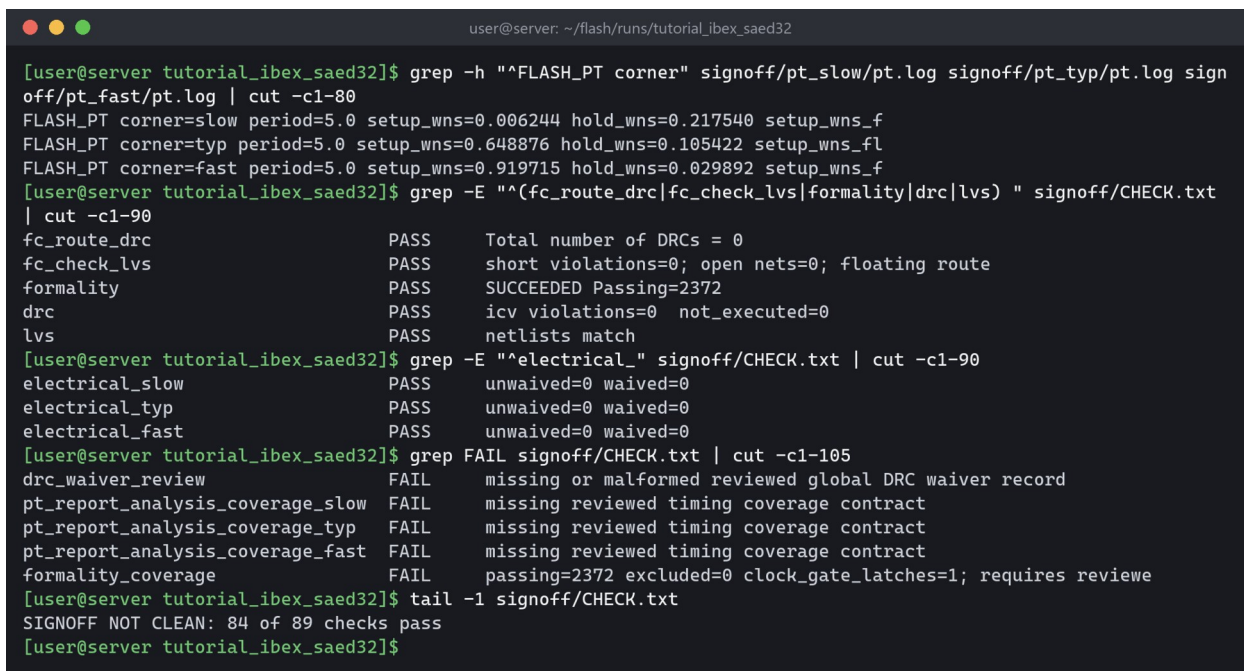
RUN=~/.flash/runs/tutorial_ibex_saed32
E=~/.flash/flow/eco_fixed.sh
LIST=manual_changes.tcl
TRY_ONLY=1 MANUAL_CHANGES=$LIST $E $RUN slow manual # try it without changing anything
TRY_ONLY=1 MANUAL_CHANGES=$LIST $E $RUN typ manual
TRY_ONLY=1 MANUAL_CHANGES=$LIST $E $RUN fast manual
MANUAL_CHANGES=$LIST $E $RUN slow manual # then apply it

```

With `TRY_ONLY=1`, the script runs PrimeTime with your list and prints any limits that would still fail. It stops there and changes nothing in the design. Fix the list until all three corners print `FLASH_MANUAL_VIOLATORS 0`. Then run the last line to apply it.

In your own run the cell names will be different. Pick them the same way. Read the cells left in `signoff/pt_slow/violators.rpt`. For each one, choose the next size up with the same logic. Test the list before you apply it. The figure above shows our list. Watch out for small cells. In this kit an X0 and an X1 cell can have the same 8 fF limit. One size up is then sometimes not enough.

End of the block run



```
user@server: ~/flash/runs/tutorial_ibex_saed32

[user@server tutorial_ibex_saed32]$ grep -h "^FLASH_PT corner" signoff/pt_slow/pt.log signoff/pt_typ/pt.log signoff/pt_fast/pt.log | cut -c1-80
FLASH_PT corner=slow period=5.0 setup_wns=0.006244 hold_wns=0.217540 setup_wns_f
FLASH_PT corner=typ period=5.0 setup_wns=0.648876 hold_wns=0.105422 setup_wns_fl
FLASH_PT corner=fast period=5.0 setup_wns=0.919715 hold_wns=0.029892 setup_wns_f
[user@server tutorial_ibex_saed32]$ grep -E "^(fc_route_drc|fc_check_lvs|formality|drc|lvs) " signoff/CHECK.txt | cut -c1-90
fc_route_drc                PASS      Total number of DRCs = 0
fc_check_lvs                PASS      short violations=0; open nets=0; floating route
formality                   PASS      SUCCEEDED Passing=2372
drc                         PASS      icv violations=0 not_executed=0
lvs                         PASS      netlists match
[user@server tutorial_ibex_saed32]$ grep -E "^electrical_" signoff/CHECK.txt | cut -c1-90
electrical_slow             PASS      unwaived=0 waived=0
electrical_typ              PASS      unwaived=0 waived=0
electrical_fast             PASS      unwaived=0 waived=0
[user@server tutorial_ibex_saed32]$ grep FAIL signoff/CHECK.txt | cut -c1-105
drc_waiver_review           FAIL      missing or malformed reviewed global DRC waiver record
pt_report_analysis_coverage_slow FAIL      missing reviewed timing coverage contract
pt_report_analysis_coverage_typ  FAIL      missing reviewed timing coverage contract
pt_report_analysis_coverage_fast  FAIL      missing reviewed timing coverage contract
formality_coverage          FAIL      passing=2372 excluded=0 clock_gate_latches=1; requires reviewe
[user@server tutorial_ibex_saed32]$ tail -1 signoff/CHECK.txt
SIGNOFF NOT CLEAN: 84 of 89 checks pass
[user@server tutorial_ibex_saed32]$
```

Figure 30. The checker after the fix loop. Timing and Formality pass now. DRC, LVS and every electrical limit pass too. The five FAIL lines left are the missing review records.

The October 1 run ended with 84 of the tutorial checker's 89 checks passing. The passing checks cover timing, crosstalk and electrical limits at all three corners. They also cover Formality, DRC and LVS. The Fusion Compiler checks pass too. These 89 checks are the tutorial checker's contract. They are not the full signoff contract that the current checker uses.

The five failures are review records. A person writes each one after reading this run's evidence. The checker makes sure it matches the run. One record covers a known wrong rule in the kit's DRC file. It confirms our layout has no errors under that rule. Some timing checks cannot apply to this block, for example a path whose input is tied to a fixed value. The record `timing_coverage.json` lists them for all three corners. The last one covers the 69 clock gates, which Formality cannot compare because they are not in the RTL. Our earlier run `w1j_saed32_resign` has finished examples of these records in its `signoff/` folder. They are `drc_waiver_review.json`, `timing_coverage.json` and `formality_coverage.json`. With those records, that run passed all 89 checks of the checker it used at the time. This tutorial stops at 84 on purpose. Signing a review record takes an engineer's own judgment. You should not copy ours.

Stage 3 and the fix loop are done when `signoff/CHECK.txt` fails only the review-record checks, or nothing once the records are signed.

Before trusting an ECO, compare its starting slack with signoff's slack. If they differ, the ECO judges timing by different rules than signoff. Its fixes will then miss. Read every warning in the Fusion Compiler log. The warning EPL-024 says cells were skipped. It was the only sign of the overlapping clock buffers. Last, let the safety net work. Round 1 broke the layout. The script caught it and put the run back.

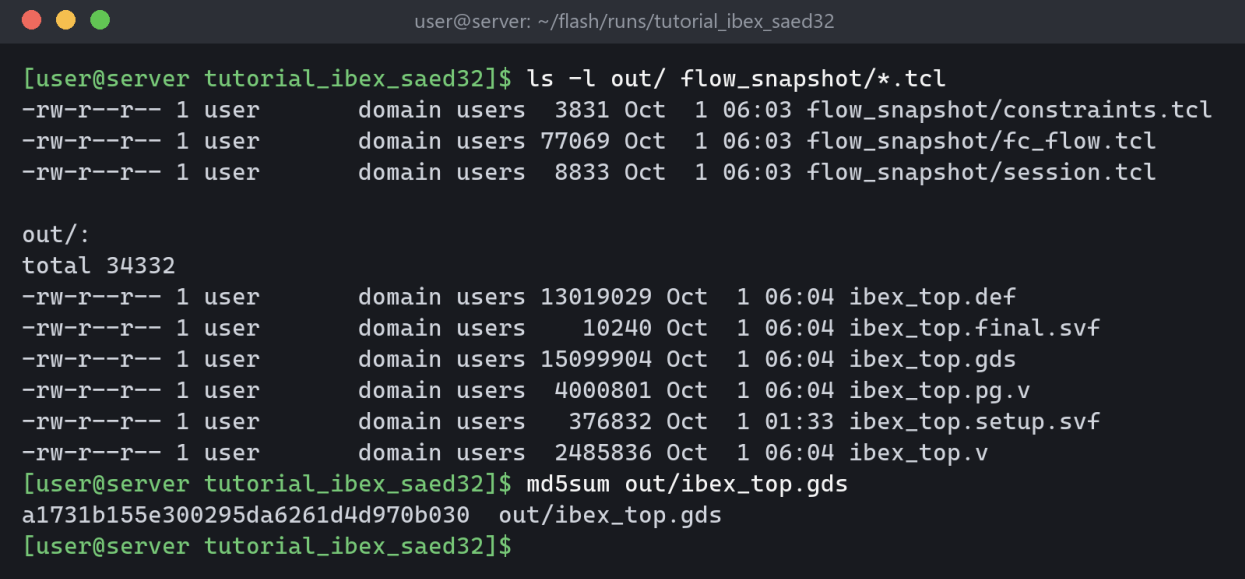
Mistakes we made on the block

Each row is a result from our earlier runs that looked right but was wrong. The last column names the check that now catches it.

What we saw	What was wrong	What caught it
Fusion Compiler met timing	crosstalk was turned off. PrimeTime failed it by 1.47 ns	PrimeTime
PrimeTime passed	the wrong wire delay file path left it with no wire delays	checking that every wire has delays
0 antenna errors	Fusion Compiler skipped every wire touching a pin	counting the skipped wires
0 routing errors after a redraw	Fusion Compiler skipped a step and left every wire open	counting open wires
Formality failed 1,971 points after a restart	the design was fine. The restart had overwritten the guide file	one guide file per session
the log said FC_EXIT=0	Fusion Compiler says 0 even after an error	looking for FLASH_FLOW_DONE
the ECO found nothing to fix	it checked timing without the 5% delay penalty	comparing it with signoff
the ECO's changed clock buffers	they were not allowed to move. They overlapped and caused 82 shorts	DRC and LVS after the ECO

Stage 4: The final files

Stage 4 freezes the set of files that the checks passed on. You can trace a result only if you can point to the exact files and to the reports the checker read. The run folder keeps both. It also keeps the scripts the run used in `flow_snapshot/`. Those scripts show how the run was made.



```
user@server: ~/flash/runs/tutorial_ibex_saed32

[user@server tutorial_ibex_saed32]$ ls -l out/ flow_snapshot/*.tcl
-rw-r--r-- 1 user      domain users  3831 Oct  1 06:03 flow_snapshot/constraints.tcl
-rw-r--r-- 1 user      domain users 77069 Oct  1 06:03 flow_snapshot/fc_flow.tcl
-rw-r--r-- 1 user      domain users  8833 Oct  1 06:03 flow_snapshot/session.tcl

out/:
total 34332
-rw-r--r-- 1 user      domain users 13019029 Oct  1 06:04 ibex_top.def
-rw-r--r-- 1 user      domain users   10240 Oct  1 06:04 ibex_top.final.svf
-rw-r--r-- 1 user      domain users 15099904 Oct  1 06:04 ibex_top.gds
-rw-r--r-- 1 user      domain users  4000801 Oct  1 06:04 ibex_top.pg.v
-rw-r--r-- 1 user      domain users   376832 Oct  1 01:33 ibex_top.setup.svf
-rw-r--r-- 1 user      domain users  2485836 Oct  1 06:04 ibex_top.v
[user@server tutorial_ibex_saed32]$ md5sum out/ibex_top.gds
a1731b155e300295da6261d4d970b030  out/ibex_top.gds
[user@server tutorial_ibex_saed32]$
```

Figure 31. The run's output files and the frozen scripts.

File	What it is for
out/ibex_top.gds	the layout
out/ibex_top.v	the netlist for timing, power and Formality
out/ibex_top.pg.v	the netlist with power connections for LVS
out/ibex_top.def	the placed and routed design for StarRC
out/ibex_top/*.svf	the guide files for Formality
rpt/	every Fusion Compiler report, stage by stage
signoff/	every checking tool's results and CHECK.txt
flow_snapshot/	the scripts this run used

`check.py` records a fingerprint of every file it judged. `signoff/manifest.json` lists the files each check needs. The fingerprints are in `signoff/provenance/`. If any of those files changes later, its checks fail until `signoff` runs again. Nobody has to lock anything by hand. Stage 4 is done when every file in the table exists. Rerun the check step with `~/flash/flow/signoff.sh $RUN check`. It must still end with the same last line.

We ran it twice

To make sure a reader can follow these steps, we ran the whole tutorial a second time on the same day. The repeat started from an empty run folder named `tutorial_ibex_saed32_repeat`. It used the commands exactly as this tutorial gives them. It ran the final `eco_fixed.sh`. It also used the shared flow as it stood that afternoon, after a teammate's changes to the checker.

The two runs came out close but not identical. Fusion Compiler does not give exactly the same result every time. Expect small differences in your own run too. The ending was the same. Both runs pass 84 of 89 checks. Only the five review records are left.

The second run needed fewer fix rounds because the final script adds buffers from the first electrical round on. None of its rounds had to be undone. Its hand list had seven cells. Trying the list first showed that two small driver cells would go over their limit, the same lesson the first run taught. That list is in the run folder as `manual_changes.tcl`.

What we compared	First run	Repeat run
cells after synthesis	8,477	8,477
cell area after synthesis, μm^2	34,634	34,634
Fusion Compiler time	14.8 minutes	15.1 minutes
clock wire errors after the clock tree	267	270
first signoff	79 of 89	79 of 89
slow setup slack at first signoff	-0.140 ns	-0.169 ns
electrical limit failures at first signoff	66, 42, 11	79, 41, 10
fix rounds kept	5	4
fix rounds undone	4	0
cells in the hand list	5	7
final result	84 of 89	84 of 89
final slow setup slack	+0.006 ns	+0.034 ns
final fast hold slack	+0.030 ns	+0.027 ns

Stage 5: Making it a chip

SAED32 is a teaching kit. Its cells and rules look real. Still, no factory builds them. A chip you can order needs a real factory's kit. We use IHP SG13G2, a 130 nm process from IHP in Germany. IHP gives its kit away for free and runs shuttles that university teams use. Other teams have already built working RISC-V chips with it. One is ETH Zurich's Croc chip. It pairs IHP's SRAM macros with CVE2, a smaller fork of Ibex.

The flow is the one you just learned, in the repository's `flow/` folder. It has a few fixes the block run did not have yet. With one of them, Fusion Compiler applies the 5% delay penalty to wire delays as well as cell delays. PrimeTime does the same. `run_fc.sh` builds the chip and `signoff.sh` checks it. From here on, every step names the repository script that runs it. `chip/make_chip.sh` runs stages 5 and 6 in one command, and stage 7 lists the commands that come after it.

The kit differs in two ways. IHP ships no IC Validator rule files. DRC and LVS use IHP's own rule files for KLayout instead. KLayout is a free layout tool. IHP also publishes only a typical wire model. Timing needs a worst and a best case for the wires too, because the metal and the insulator vary a little from chip to chip. We built the worst and best wire models ourselves from ranges in IHP's process specification. The ranges cover sheet resistance, via resistance and insulator dielectric constant. Sheet resistance says how strongly a metal layer resists current. The dielectric constant of the insulator sets how much capacitance a wire has to its neighbors. The specification lists only targets for metal and insulator thickness. We kept those at their typical values. Section 6.1 shows one rough test of how much this matters.

5.1 The chip

The chip is called `flash_chip`. Inside is the Ibex processor with two 4 KiB memories. One holds the program and one holds data. A serial loader lets a test board write a program into memory through three pins. The program can set eight output pins. The chip also has a scan chain for factory testing. Around the core sit 25 pads. There are 8 inputs and 9 outputs. Four pads carry the 1.2 V core supply and its ground. Four more carry the 3.3 V pad supply and its ground.

The run's `run_cfg.tcl` sets all of that, as the block's file did. The new lines name the pads and set where the two memories go. They also say which test pins to hold off during normal timing and how to build the scan chain. One more line matters. `PNR_MARGIN 0.10` makes Fusion Compiler aim for an 18 ns clock, a period 10% shorter than the 20 ns we sign off at. On the fresh build of the repository that this stage shows, the finished chip kept +0.71 ns on its slowest path.

```

user@server: ~/flash/runs/test1

[user@server test1]$ grep -E "^set (DESIGN|PDK|CLK_PORT|PERIOD_NS|PNR_MARGIN|UTIL|MACROS|DFT_SETUP|SDC_EXTRA|OUT)" run_cfg.tcl | sed "s#\$HOME/tt#~#g" | cut -c1-104
set DESIGN      flash_chip
set CLK_PORT    clk
set PERIOD_NS   20.0
set PNR_MARGIN  0.10
set UTIL        0.5
set OUT         ~/flash/runs/test1
set PDK         ihp_sg13g2
set MACROS      {u_soc/u_imem 244 810.54 R0 u_soc/u_dmem 684.64 810.54 R0}
set DFT_SETUP   {~/flash/flow/dft_setup.tcl}
set SDC_EXTRA    ~/flash/chip/bist_constants.sdc
[user@server test1]$

```

Figure 32. The main settings of the run, from a fresh build of the repository in a run named test1. Every screenshot in stages 5 to 7 comes from that build.

```

bash ~/flash/chip/new_run.sh my_chip      # makes ~/flash/runs/my_chip/run_cfg.tcl
RUN=~/flash/runs/my_chip
export RUN_CFG=$RUN/run_cfg.tcl FLASH_TOOLS=2026
STOP_AFTER=synth ~/flash/flow/run_fc.sh $RUN      # synthesis and scan chain
START_AT=floorplan ~/flash/flow/run_fc.sh $RUN    # to final, ends with FLASH_FLOW_DONE

```

new_run.sh copies chip/run_cfg.tcl into the new folder and sets OUT and the paths to your home folder. RUN_CFG names the settings file. FLASH_TOOLS=2026 picks the 2026.03 release of the Synopsys tools. That release is also the default for a new run. The build runs in two sessions so you can check synthesis and the scan chain before placing anything. After the first session, the log must say FLASH_IHP_PADS_POST_DFT 25. That means all 25 pads survived scan insertion. Scan insertion is the step that builds the scan chain. It is also called DFT, short for design for test.

5.2 Pads, seal ring and fill

Fusion Compiler places the pads and routes the chip. After that, finish.sh adds three things the factory needs. Bond pads are the metal openings that a wire from the package is welded to. The seal ring goes around the edge with a 75 μm gap. Metal fill goes into every empty area. The factory polishes each layer flat. A layer with too little or too much metal in one spot polishes unevenly. IHP requires between 35% and 60% metal on Metal1 to Metal5. On the two thick top metals it requires between 25% and 70%.

In the command below, 75 is the seal ring gap in μm . bondpads asks for bond pads on every pad. METAL_FILL=mid picks IHP's own fill pattern. The pattern sits 1.0 μm from existing metal on Metal2 to Metal5. The other settings left too little or too much metal in some parts of earlier test chips. After the run, chip/density.txt shows each layer's average over the whole die. Every layer must fall inside IHP's limits. IHP's rules also check density in smaller windows across the chip. The whole-chip DRC runs those rules and must be clean too.

```

METAL_FILL=mid bash ~/flash/flow/finish.sh $RUN 75 bondpads
cat $RUN/chip/density.txt

```

```
user@server: ~/flash/runs/test1

[user@server test1]$ grep -h "^FLASH_IHP_PADS_POST_DFT\|^FLASH_FLOW_DONE" fc_setup.log fc_floorplan.log
FLASH_IHP_PADS_POST_DFT 25
FLASH_FLOW_DONE
[user@server test1]$ cat chip/density.txt
bbox (0,0;1708,1753) area 2994124 um2
Activ      47.2%    0:29.6% 22:17.6%
GatPoly    17.7%    0:7.0% 22:10.7%
Metal1     49.7%    0:29.8% 22:19.9%
Metal2     44.6%    0:8.3% 22:36.2%
Metal3     53.5%    0:31.3% 22:22.2%
Metal4     54.6%    0:32.4% 22:22.2%
Metal5     55.0%    0:28.1% 22:26.9%
TopMetal1  51.5%    0:24.8% 22:26.7%
TopMetal2  50.5%    0:21.5% 22:29.1%
[user@server test1]$ grep -h "Number of DRC errors" chip/drc.log chip/precheck.log
Number of DRC errors for maximum rule set: 0
Number of DRC errors for maximum rule set: 0
[user@server test1]$
```

Figure 33. All 25 pads survived scan insertion and the flow finished. Every metal layer lands between 44.6% and 55.0%, inside IHP's limits, and whole-chip DRC and IHP's pre-check find 0 errors.

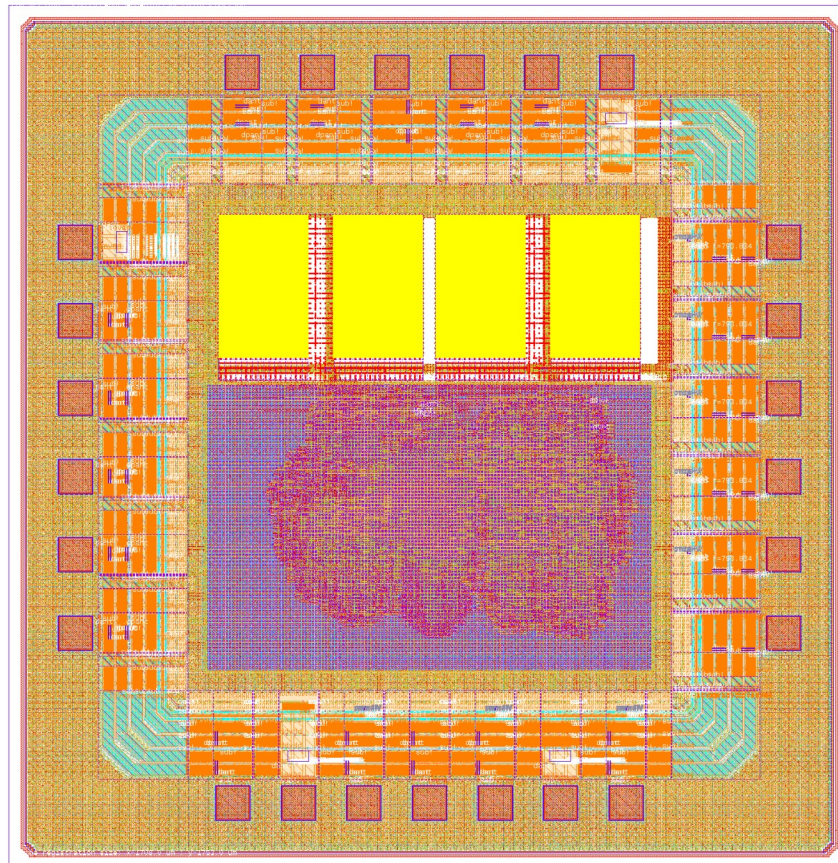


Figure 34. The finished chip in KLayout, 1708 by 1753 μm . The seal ring runs around the edge, the 25 square bond pads sit on the pads, the two memories are yellow, and fill covers every empty area.

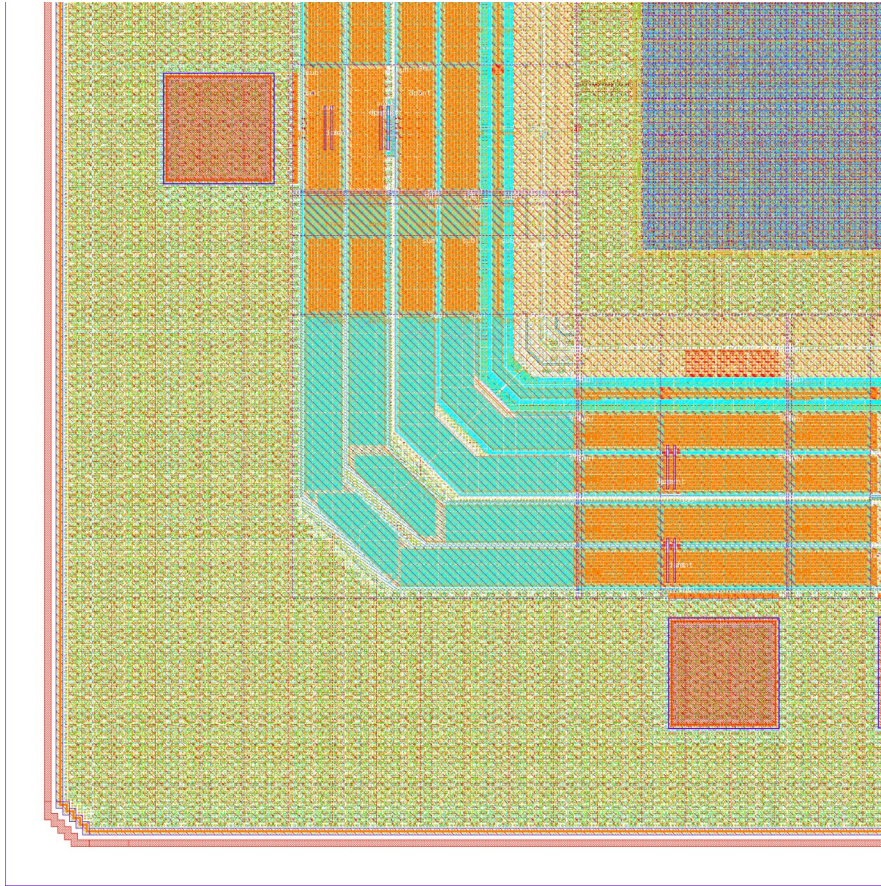


Figure 35. The lower left corner, 560 μm across. From the outside in: the seal ring, two bond pads, the pad ring's corner cell with its supply rails, and the edge of the core.

5.3 Checking the whole chip

Fill and the seal ring change the layout. So DRC and LVS run again on the finished chip. IHP also has a pre-check. It is a shorter rule set that IHP runs on every design before accepting it. `finish.sh` runs whole-chip DRC and the pre-check. Whole-chip LVS runs as part of `signoff.sh`, which section 6.1 shows. DRC, the pre-check and LVS must all be clean.

```

user@server: ~/flash/runs/test1/signoff

[user@server signoff]$ grep -h "Number of DRC errors" drc.log
Number of DRC errors for maximum rule set: 0
[user@server signoff]$ grep -h "Congratulations" lvs.log lvs_chip/lvs.log | sed 's/^.*INFO : //'
Congratulations! Netlists match.
Congratulations! Netlists match.
[user@server signoff]$ grep -E "^(drc|lvs|lvs_completion|chip_lvs_completion) " CHECK.txt | cut -c1-104
drc                PASS      klayout violations=0 recommended_only=0 chip_level=GFi
lvs                 PASS      netlists match
lvs_completion      PASS      latest match verdict and successful exit required
chip_lvs_completion PASS      latest match verdict and successful exit required
[user@server signoff]$

```

Figure 36. KLayout with IHP's rules at signoff. Block DRC finds 0 errors. LVS matches for the block and for the whole chip with its pad ring.

IHP's own memory and pad cells fail IHP's own LVS rule file. They fail even when checked alone with no design around them. The problem is inside IHP's cells, not in how our design uses them. We could not find out why, so whole-chip LVS treats them as sealed boxes. It checks every wire that connects to them but not their insides. This LVS does not cover the insides of these cells. Other teams have built chips with these cells. Their results do not count as an LVS check of the insides for our chip.

Stage 5 is done when the log has FLASH_IHP_PADS_POST_DFT 25 and FLASH_FLOW_DONE. Every metal layer in chip/density.txt must also be inside IHP's limits. Whole-chip DRC and IHP's pre-check must be clean.

Stage 6: Proving it works

A clean layout is not enough. This stage checks each way the chip can fail in real life:

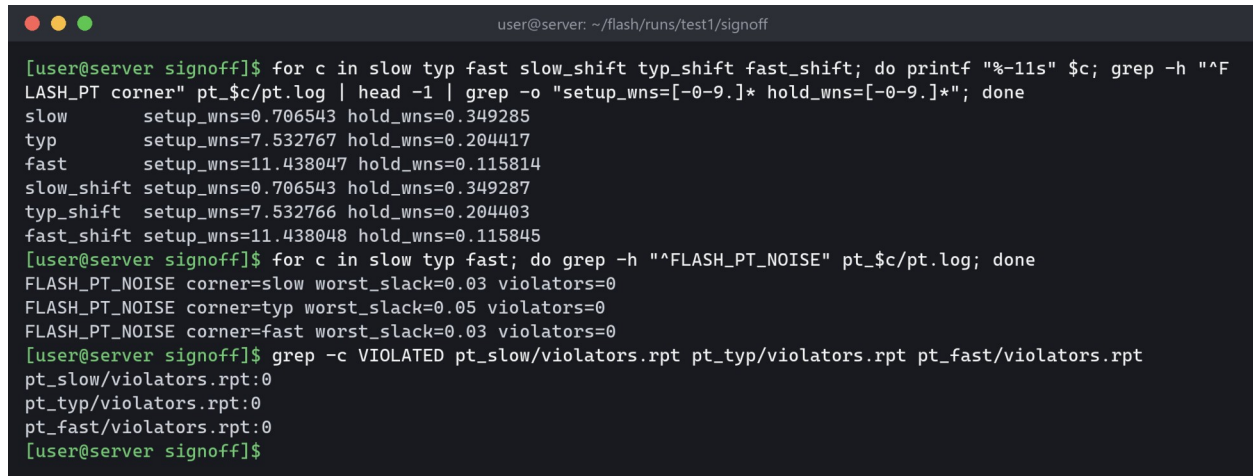
- It is too slow.
- It is too fast.
- Its logic is wrong.
- A memory is bad.
- The factory left a broken transistor.
- A signal from the board arrives at the wrong moment.
- The power grid sags.

Each check has its own section.

6.1 Timing at every corner, in both modes

The chip has two modes. In normal mode it runs programs. In scan mode the tester shifts patterns through the scan chain. PrimeTime checks both modes at all three corners. That makes six runs. Each run uses the 5% delay penalty and checks crosstalk. The two export lines point StarRC at the worst and best wire models we built from IHP's process specification.

```
export RC_CMAX_GRID=~/.flash/ihp/rc/w6_spec_sensitivity/rcmax/sg13g2_spec_rcmax.nxtgrd
export RC_CMIN_GRID=~/.flash/ihp/rc/w6_spec_sensitivity/rcmin/sg13g2_spec_rcmin.nxtgrd
# runs StarRC, PrimeTime six times, Formality, PrimePower, DRC, LVS and whole-chip LVS
~/flash/flow/signoff.sh $RUN
```



```
user@server: ~/flash/runs/test1/signoff

[user@server signoff]$ for c in slow typ fast slow_shift typ_shift fast_shift; do printf "%-11s" $c; grep -h "^FLASH_PT corner" pt_${c}/pt.log | head -1 | grep -o "setup_wns=[-0-9.]* hold_wns=[-0-9.]*"; done
slow      setup_wns=0.706543 hold_wns=0.349285
typ       setup_wns=7.532767 hold_wns=0.204417
fast      setup_wns=11.438047 hold_wns=0.115814
slow_shift setup_wns=0.706543 hold_wns=0.349287
typ_shift setup_wns=7.532766 hold_wns=0.204403
fast_shift setup_wns=11.438048 hold_wns=0.115845
[user@server signoff]$ for c in slow typ fast; do grep -h "^FLASH_PT_NOISE" pt_${c}/pt.log; done
FLASH_PT_NOISE corner=slow worst_slack=0.03 violators=0
FLASH_PT_NOISE corner=typ worst_slack=0.05 violators=0
FLASH_PT_NOISE corner=fast worst_slack=0.03 violators=0
[user@server signoff]$ grep -c VIOLATED pt_slow/violators.rpt pt_typ/violators.rpt pt_fast/violators.rpt
pt_slow/violators.rpt:0
pt_typ/violators.rpt:0
pt_fast/violators.rpt:0
[user@server signoff]$
```

Figure 37. Setup and hold slack in all six runs after the final signoff. Crosstalk noise and the electrical limits have 0 violations.

Corner	Setup, normal	Setup, scan	Hold, normal	Hold, scan
slow, 1.08 V and 125 C	+0.707 ns	+0.707 ns	+0.349 ns	+0.349 ns
typical, 1.2 V and 25 C	+7.533 ns	+7.533 ns	+0.204 ns	+0.204 ns
fast, 1.32 V and -40 C	+11.438 ns	+11.438 ns	+0.116 ns	+0.116 ns

All six runs pass setup and hold with 0 electrical violations. Remember that we built the worst and best wire models ourselves. To see whether that could hide a failure, on our first build of this chip we doubled the delay penalty from 5% to 10%. We then ran the slow and fast corners again. Slow setup dropped to +0.053 ns and fast hold to +0.049 ns. Both still pass. This tests only the assumptions we chose. It does not cover every possible model error. It also does not replace wire corners qualified by the factory.

6.2 Adding timing margin

The first signoff of the fresh build passed setup with +1.142 ns. Its fast hold slack was only +0.037 ns in both modes. In the lab you can fix a setup failure by slowing the clock. You cannot fix a hold failure that way. A slack of +0.037 ns passes, but it leaves almost no room for the small differences between real chips. So we add margin. One more problem showed at every corner. A flip-flop uses only its inverted output, so its other output drives nothing and is under its smallest allowed load.

Every fix can break something at another corner. PrimeTime fixes hold at the fast corner. While it does, it cannot see the slow corner. On our first build of this chip, its first list of hold fixes would have turned slow setup from +1.216 ns into -2.65 ns. Without a setup limit, its noise fix alone turned slow setup into -0.154 ns. We therefore tested every list of changes at all three corners and both modes before Fusion Compiler applied it. We dropped any changed cell that pushed a slow path under +0.6 ns. Then we tested the list again.

The script `chip/margin_fix.sh` runs one round of these steps unattended and keeps its work in `$RUN/margin/roundN`. `make_chip.sh` runs up to three rounds, until timing, noise and electrical limits pass and fast hold reaches +0.09 ns. The box shows the steps of a round in order. The lines that start with # describe steps the script runs for you.

```
bash ~/flash/chip/margin_fix.sh $RUN
```

1. PrimeTime proposes noise, load and hold fixes and changes nothing.
2. The script tests the joined list in six PrimeTime runs, three corners in two modes.
3. It drops each changed cell that drives an electrical violation or leaves a path under its floor. The floor is +0.6 ns at slow and +0.2 ns at typ and fast. It tests the shorter list again until it drops nothing.
4. `chip/margin/targeted.py` adds fixes for what is left, and the list is tested again.
5. Fusion Compiler applies the list. The script finishes the chip and signs it off again. It undoes the round if any check that passed before now fails.

The comments at the top of `chip/margin_fix.sh` describe the same steps, and the files it uses are in `chip/margin/`.

Step 4 is what we first did by hand on our first build. After PrimeTime's own lists, two noise victims, three memory inputs short of the hold floor and one driver over its transition limit were left. Each victim's driver got a buffer after it. Each memory input got a delay cell in front of it. The driver got one size larger. `targeted.py` now makes the same three kinds of change on its own.

```

user@server: ~/flash/runs/test1/margin/round1

[user@server round1]$ grep -E "cand0 changes|FILTER|targeted pass|apply" ../../make_chip.log | sed -E 's/^20[0-9-]+ [0-9:]+ //'
cand0 changes: 298
FILTER bad_cells 2 dropped 2 kept 296
targeted pass 1: 14 changes
targeted pass 2: 10 changes
apply 320 changes
[user@server round1]$ sed -E 's/FLASH_EVAL //; s/ changed=[0-9]+//; s/([0-9]\.[0-9]{3})[0-9]+/\1/g' cand0.eval
corner=fast mode=func bad=0 drc_viol=0 noise_viol=0 setup_wns=11.468 hold_wns=0.142
corner=slow mode=func bad=2 drc_viol=14 noise_viol=0 setup_wns=0.802 hold_wns=0.389
corner=typ mode=func bad=0 drc_viol=0 noise_viol=0 setup_wns=7.579 hold_wns=0.240
corner=fast mode=shift bad=0 drc_viol=0 noise_viol=0 setup_wns=11.468 hold_wns=0.142
corner=slow mode=shift bad=2 drc_viol=14 noise_viol=0 setup_wns=0.802 hold_wns=0.389
corner=typ mode=shift bad=0 drc_viol=0 noise_viol=0 setup_wns=7.579 hold_wns=0.240
[user@server round1]$

```

Figure 38. Round 1 on the fresh build. PrimeTime proposed 298 changes. At the slow corner 2 of them left a path under the floor, and the test dropped both.

```

user@server: ~/flash/runs/test1/margin/round1

[user@server round1]$ head -3 targeted1.tcl | cut -c1-80
insert_buffer [get_pins {u_soc/u_imem/A_DIN[0]}] sg13g2_dlygate4sd1_1 -new_net_n
insert_buffer [get_pins {u_soc/u_dmem/A_BM[8]}] sg13g2_dlygate4sd1_1 -new_net_na
insert_buffer [get_pins {u_soc/u_dmem/A_BM[9]}] sg13g2_dlygate4sd1_1 -new_net_na
[user@server round1]$ grep -h "^size_cell" targeted1.tcl | head -2
size_cell {u_soc/u_core/u_ibex_core/A4312} {sg13g2_nand2_2}
size_cell {u_soc/u_core/u_ibex_core/A4318} {sg13g2_nand2_2}
[user@server round1]$ sed -E 's/FLASH_EVAL //; s/ changed=[0-9]+//; s/([0-9]\.[0-9]{3})[0-9]+/\1/g' cand3.eval
corner=fast mode=func bad=0 drc_viol=0 noise_viol=0 setup_wns=11.468 hold_wns=0.113
corner=slow mode=func bad=0 drc_viol=0 noise_viol=0 setup_wns=0.802 hold_wns=0.339
corner=typ mode=func bad=0 drc_viol=0 noise_viol=0 setup_wns=7.579 hold_wns=0.202
corner=fast mode=shift bad=0 drc_viol=0 noise_viol=0 setup_wns=11.468 hold_wns=0.113
corner=slow mode=shift bad=0 drc_viol=0 noise_viol=0 setup_wns=0.802 hold_wns=0.339
corner=typ mode=shift bad=0 drc_viol=0 noise_viol=0 setup_wns=7.579 hold_wns=0.202
[user@server round1]$ cat gate.txt
MARGIN_GATE PASS
[user@server round1]$

```

Figure 39. The targeted fixes, and the final test of all 320 changes. Fast hold is +0.113 ns, slow setup keeps +0.802 ns, and no corner has a bad cell, an electrical violation or a noise violation.

On the fresh build, dropping the 2 bad cells took fast hold back under the floor, so the targeted fixes ran. They put delay cells in front of memory inputs and made 5 drivers one size larger. Fusion Compiler applied the 320 changes and swapped the flip-flop for one without the unused output, and the round was kept with 123 checks passing against 117 before. A second, smaller round of 23 changes followed, because of a bug in how make_chip.sh decided a round was needed, now fixed. Fast hold went from +0.037 ns to +0.116 ns, and slow setup kept +0.707 ns.

6.3 Running real programs on the gates

Timing checks show the chip is fast enough. A gate-level simulation shows it computes the right answers before the chip is built. It covers only the programs we tested. It is also only as good as the simulation models. VCS runs the final netlist with every cell and wire delay from PrimeTime's SDF files. It runs at the slow, typical and fast corners. A testbench loads each program through the serial pins, starts the processor and reads the output pins. A program reports pass by setting the outputs to 0x80.

Program	What it checks
main	the original self-check program
t_isa	every RV32I instruction type against known answers. RV32I is the basic set of instructions every 32-bit RISC-V processor runs
t_muldiv	each RV32M multiply and divide instruction. It covers chosen normal cases, divide by zero and signed overflow
t_mem	a March C- test of the lower 960 data-memory words. March C- is a standard memory test that writes and reads every word in a fixed order of upward and downward passes. It also runs address and checkerboard patterns and reads eight constants from program memory. It tests the outputs too. The top 256 bytes hold the stack and are not march-tested
t_work	a prime sieve, a sort and a Fibonacci table checked against known answers
t_muldiv_plant	a control built to fail. It must report fail. That proves a failure would show up

```
bash ~/flash/chip/gls.sh $RUN      # all six programs at slow, typical and fast, in $RUN/gls
```

```

user@server: ~/flash/runs/test1/gls

[user@server gls]$ for c in slow typ fast; do echo "== $c"; cut -c1-96 $c/summary.txt; done
== slow
main want=PASS got=[TB_RESULT PASS gpio=80 time=1311120000] OK
t_isa want=PASS got=[TB_RESULT PASS gpio=80 time=1976880000] OK
t_muldiv want=PASS got=[TB_RESULT PASS gpio=80 time=1017700000] OK
t_mem want=PASS got=[TB_RESULT PASS gpio=80 time=3287520000] OK
t_work want=PASS got=[TB_RESULT PASS gpio=80 time=1966700000] OK
t_muldiv_plant want=FAIL got=[TB_RESULT FAIL gpio=a0 time=1266840000] OK
== typ
main want=PASS got=[TB_RESULT PASS gpio=80 time=1311120000] OK
t_isa want=PASS got=[TB_RESULT PASS gpio=80 time=1976880000] OK
t_muldiv want=PASS got=[TB_RESULT PASS gpio=80 time=1017700000] OK
t_mem want=PASS got=[TB_RESULT PASS gpio=80 time=3287520000] OK
t_work want=PASS got=[TB_RESULT PASS gpio=80 time=1966700000] OK
t_muldiv_plant want=FAIL got=[TB_RESULT FAIL gpio=a0 time=1266840000] OK
== fast
main want=PASS got=[TB_RESULT PASS gpio=80 time=1311120000] OK
t_isa want=PASS got=[TB_RESULT PASS gpio=80 time=1976880000] OK
t_muldiv want=PASS got=[TB_RESULT PASS gpio=80 time=1017700000] OK
t_mem want=PASS got=[TB_RESULT PASS gpio=80 time=3287520000] OK
t_work want=PASS got=[TB_RESULT PASS gpio=80 time=1966700000] OK
t_muldiv_plant want=FAIL got=[TB_RESULT FAIL gpio=a0 time=1266840000] OK
[user@server gls]$

```

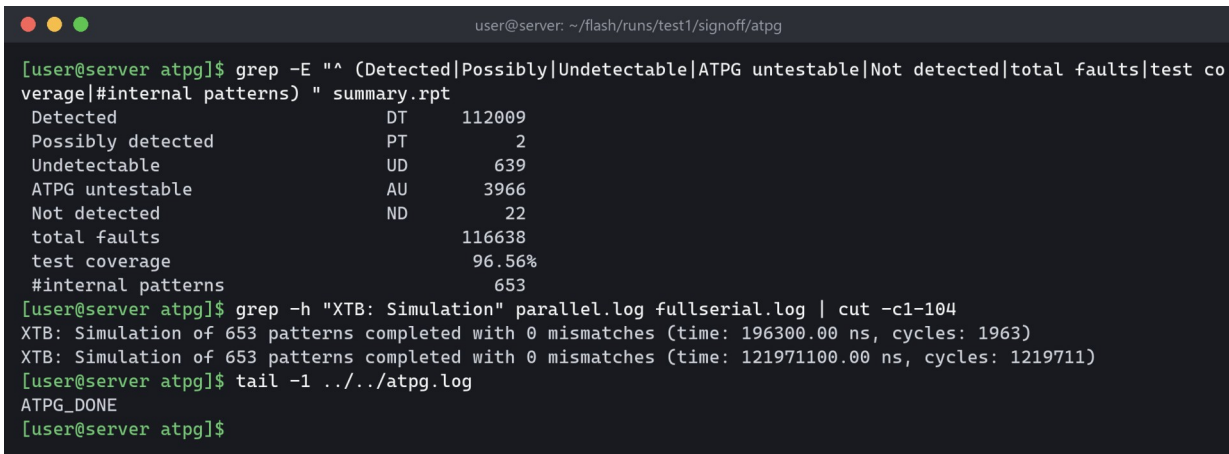
Figure 40. Every program at every corner. All five real programs pass with 0x80 on the GPIO pins, and t_muldiv_plant fails as it must, at slow, typ and fast.

At the fast corner the memory's simulation model prints 221,981 timing warnings across the six programs. Every one comes from the same check. The model tests its address pins against a fixed 1 ns hold time. IHP's characterized library says 0.32 to 0.34 ns. PrimeTime meets that. Every program still passes at the fast corner. The full logs are in \$RUN/gls/fast/.

6.4 Testing for factory defects

Some chips come out of the factory with a broken transistor or a wire that did not form. A tester finds them with scan patterns. TestMAX makes the patterns, and VCS replays them twice on the final netlist. The parallel replay is quick because it loads each pattern straight into the flip-flops. The serial replay shifts every bit in and out through the scan pins. A real tester does the same. Both replays run with zero delay and timing checks off. The patterns find stuck-at faults. A stuck-at fault is a wire stuck at 0 or 1. The patterns do not test for faults that only make a path slow.

```
bash ~/flash/chip/atpg.sh $RUN      # patterns and both replays, in $RUN/signoff/atpg
```

A terminal window showing the execution of a test script. The user runs a grep command to filter a summary report, showing statistics for detected faults, test coverage, and simulation results. The output indicates that 112,009 faults were detected out of 116,638 modeled faults, with a test coverage of 96.56%. Both parallel and full serial simulations completed with 0 mismatches.

```
user@server: ~/flash/runs/test1/signoff/atpg
[user@server atpg]$ grep -E "^(Detected|Possibly|Undetectable|ATPG untestable|Not detected|total faults|test coverage|#internal patterns) " summary.rpt
Detected                DT      112009
Possibly detected       PT         2
Undetectable            UD       639
ATPG untestable         AU      3966
Not detected            ND        22
total faults            116638
test coverage           96.56%
#internal patterns      653
[user@server atpg]$ grep -h "XTB: Simulation" parallel.log fullserial.log | cut -c1-104
XTB: Simulation of 653 patterns completed with 0 mismatches (time: 196300.00 ns, cycles: 1963)
XTB: Simulation of 653 patterns completed with 0 mismatches (time: 121971100.00 ns, cycles: 1219711)
[user@server atpg]$ tail -1 ../../atpg.log
ATPG_DONE
[user@server atpg]$
```

Figure 41. 653 stuck-at patterns detect 112,009 of 116,638 modeled faults. No test can detect 639 of those faults, because no input can make them change an output. Leaving those 639 out gives 96.56% test coverage. Both replays show 0 mismatches.

Scan patterns do not reach inside the memories. The `t_mem` program exercises them instead and passes at every corner. It is not a full memory test. It leaves out the stack area of data memory and only reads program memory.

6.5 Signals from the board

The serial loader's pins and the reset come from the test board. The board has its own timing. If a board signal changes right at a clock edge, a flip-flop can be caught halfway. For a moment its output sits between 0 and 1 before it settles. Logic that reads it during that moment can go wrong. The fix is a synchronizer, which is two flip-flops in a row. The second one waits one tick, which gives the first one time to settle. SpyGlass CDC checks that every board signal has one. `chip/cdc.sh` writes the project file `cdc.prj`. It names the design files and runs three checks. They are the setup check, the structure check and the full check. The constraints file `chip/flash_soc.sgdc` puts the loader pins on a separate board clock. SpyGlass checks the RTL of the core without the pad ring. The design applies the reset at once and releases it through its own synchronizer.

```
bash ~/flash/chip/cdc.sh $RUN      # setup_check, verify_struct and verify reports in $RUN/cdc
```

```

user@server: ~/flash/runs/test1/cdc

[user@server cdc]$ grep -E "Ac_sync0[12]|Ar_sync01|Ar_syncdeassert01" verify.rpt | sed -E 's/^\[[0-9]+\] +([A-Za-z_0-9]+) +Info +[^\ ]+ +[0-9]+ +[0-9]+ +/\1: /; s/flash_soc\./g; s/ +/ /g' | cut -c1-104 | head -5
Ac_sync02: Synchronized Crossing: destination flop sclk_s[0], clocked by clk, source primary input sclk,
Ac_sync02: Synchronized Crossing: destination flop sdi_s[0], clocked by clk, source primary input sdi, c
Ac_sync02: Synchronized Crossing: destination flop load_s[0], clocked by clk, source primary input load_
Ar_sync01: Reset signal 'rst_n' for 'clear' pin of flop 'sclk_s[0]', clocked by 'clk', is synchronized b
Ar_sync01: Reset signal 'rst_n' for 'clear' pin of flop 'u_core.core_busy_q[0]', clocked by 'clk', is sy
[user@server cdc]$ grep -c "Ac_unsync" verify.rpt
0
[user@server cdc]$

```

Figure 42. SpyGlass finds a synchronizer on all three loader inputs and on the reset. It finds 0 unsynchronized crossings.

SpyGlass also found one rule for whoever builds the test board. The data pin and the clock pin of the loader pass through separate synchronizers. The data pin must therefore not change near a rising edge of the loader clock. Change it while the loader clock is low and at least 4 chip clock cycles from either edge.

6.6 Supply voltage drop and electromigration

RedHawk-SC checks the power grid as in section 3.7. This time it uses the chip's real pads. Current enters the core through straps from the pad ring. We ran two bounds. The pessimistic one feeds the chip only through the straps next to the four core supply pads. The optimistic one feeds it through every strap. We also reran the pads case with switching activity recorded while the `t_work` program ran on the gate netlist. That run replaces the 10% activity guess. The EM check compares the current in every power wire and via with IHP's published limits for 11 years at 105 C. It checks only the steady DC current in the power grid. We did not check signal and clock wires for EM.

```

export ANSYSLMD_LICENSE_FILE=<port>@<your Ansys license server>
bash ~/flash/chip/ir/ir.sh $RUN
# 1. runs t_work on the gate netlist and records its switching activity
# 2. writes the five cases: pads, straps, the activity case and the two controls
# 3. runs RedHawk-SC on each and prints the worst cell and worst EM, in $RUN/ir

```

```

user@server: ~/flash/runs/test1

[user@server test1]$ grep worst_cell ir.log | sed -E 's/worst_cell=([0-9.]+) [^\ ]+/worst=\1 V/; s/em_wire_max=/E
M wire %=/; s/em_via_max=/via %=/; s/em_fail=/fails=/'
pads worst=1.173 V EM wire %=62.12 via %=35.64 fails=0
straps worst=1.19 V EM wire %=20.3 via %=12.89 fails=0
saif worst=1.187 V EM wire %=26 via %=17.69 fails=0
emcontrol worst=1.173 V EM wire %=6.212e+04 via %=3.564e+04 fails=5000
defect worst=1.167 V EM wire %=69.98 via %=34.73 fails=0
[user@server test1]$

```

Figure 43. Worst cell voltage on the 1.2 V supply and worst EM for each case. The two control runs must look worse than the real cases, and both do.

Case	Worst cell	Drop	Worst EM
pads only, 10% activity guess	1.173 V	27 mV, 2.3%	62.1% of the limit
every strap, 10% activity guess	1.190 V	10 mV	20.3%
pads only, real program activity	1.187 V	13 mV, 1.1%	26.0%
control with the grid weakened on purpose	1.167 V	33 mV	must be worse and is
control with EM limits cut 1000 times			all 5,000 listed rows fail as they must

We compare the drop against an allowance of 10% of the supply. The 10% is our own project choice and not an IHP limit. The worst case here is 2.3%. The model leaves out the pad ring's own rails and any current that changes quickly. A hand calculation for the package assumes 0.1 ohm of bond wire and lead and about 8.1 mA through each core supply pad. That adds just under 1 mV on the supply and about the same on ground. The worst drop then grows from 27 mV to about 29 mV, or 2.4%. These checks use approximate current models. They do not count as a qualified dynamic voltage-drop signoff.

Stage 6 is done when all of these hold:

- All six PrimeTime runs pass.
- Every program passes at all three corners and the planted failure fails.
- Both scan replays show 0 mismatches.
- SpyGlass finds 0 unsynchronized crossings.
- No real case breaks an EM limit.
- The worst drop stays inside the 10% allowance.
- Both control runs look worse than the real cases.

Stage 7: Ready for the factory

Stage 7 closes the last open items and packages the design. An engineer signs a review record for each thing a tool cannot judge. After that, signoff runs again. Our script then saves the final GDS in the form IHP asks for. We check that file once more.

7.1 Review records

Some things a tool can report but cannot judge. PrimeTime says two memory clock pins have no clock, and it is right. Those pins belong to the memory's built-in self-test port. This chip does not use that port. A tool can only check what it was set up to check. A person has to confirm that and sign off on it. `check.py` keeps failing these checks until a signed record with the evidence in it exists.

Record	What the engineer checked
timing_waivers.json	the two unlocked pins are exactly the memories' self-test clocks. Every self-test pin is tied low in the netlist
timing_coverage.json	every timing check PrimeTime skips. These are test pins held off in each mode, reset pulse widths and constants. PrimeTime skips no flip-flop for lack of a clock
formality_coverage.json	the one point Formality leaves out. That point is the scan output. It is used only in test mode. The scan test covers it
atpg/evidence.json	the 96.56% coverage and the replays. Every TestMAX warning traces to a simulation model. None traces to the design

`make_chip.sh` ends by writing unsigned drafts of these records to `$RUN/review`, with `SUMMARY.txt` and `EVIDENCE.txt` beside them. Read both. Sign only if the evidence agrees with the basis `sign.py` lists. If `SUMMARY.txt` names an UNEXPECTED pin, do not sign. Find out why that pin has no clock first.

```
python3 ~/flash/chip/review/sign.py $RUN "Your Name"
bash ~/flash/chip/final_signoff.sh $RUN          # all of signoff again, ends with SIGNOFF CLEAN
```

`check.py` ties the records to the PrimeTime and Formality results. New records made those results out of date. That is why the whole signoff runs again from the start after signing.

```
user@server: ~/flash/runs/test1/signoff

[user@server signoff]$ grep -E "^(fc_flow|chip_drc|chip_precheck|chip_lvs_completion|formality|atpg_coverage|formality_coverage|electrical_slow|pt_slow|noise_slow|timing_waivers|pt_report_analysis_coverage_slow) " CHECK.txt
| cut -c1-104
fc_flow                PASS      done=True errors=0
pt_slow                PASS      period=20.0ns setup_wns=0.707 hold_wns=0.349 spef_nets=
noise_slow            PASS      worst_slack=0.03 violators=0
formality              PASS      SUCCEEDED Passing=2188
chip_drc               PASS      klayout violations=0 recommended_only=0
chip_precheck          PASS      klayout violations=0 recommended_only=0
electrical_slow        PASS      unwaived=0 waived=0
pt_report_analysis_coverage_slow PASS      reviewed reasons=conditional_arc:304,constant_at_pin:6,
formality_coverage     PASS      passing=2188 excluded=1 clock_gate_latches=0; requires
atpg_coverage          PASS      patterns=653 test_coverage=96.56 total_faults=116638 St
chip_lvs_completion    PASS      latest match verdict and successful exit required
[user@server signoff]$ grep -c " PASS " CHECK.txt
139
[user@server signoff]$ tail -1 CHECK.txt
SIGNOFF CLEAN: 139 of 139 checks pass
[user@server signoff]$
```

Figure 44. A sample of the final CHECK.txt, the count of passing checks and its last line. It says SIGNOFF CLEAN with 139 of 139 checks passing on October 5, 2026.

7.2 The release package

IHP takes designs as a folder with a fixed layout. IHP's `gen_structure.py` script makes that layout. The folder holds the final GDS, the netlist, documents and the design files. IHP requires exact save settings for the GDS. These are a 1 nm grid, the library name LIB and at most 8,000 corners per shape. A corner here is a vertex of a shape. A long, jagged shape can have thousands. Our script builds the folder. It renames the top cell to RVSoC8787, the IP name IHP's script generated. It then saves the GDS with those settings.

```
bash ~/flash/chip/package/package.sh $RUN
# 1. fills $RUN/package/IHP__RVSoC8787: GDS with IHP's save settings, netlists, design files
# 2. compares the release GDS with the signed-off chip, shape by shape and label by label
# 3. runs IHP's pre-check on the exact file that would be sent, and ends with PACKAGE_OK
```

```

user@server: ~/flash/runs/test1/package

[user@server package]$ ls IHP__RVSoC8787 IHP__RVSoC8787/release/v.1.0.0/gds IHP__RVSoC8787/release/v.1.0.0/netlist
IHP__RVSoC8787:
doc
README.md
release
RVSoC8787-main

IHP__RVSoC8787/release/v.1.0.0/gds:
RVSoC8787.gds

IHP__RVSoC8787/release/v.1.0.0/netlist:
flash_chip.v
RVSoC8787.cdl
[user@server package]$ grep -h "^LAYERS\|^TEXTS" xor.txt texts.txt
LAYERS 59 DIFFERING 8
TEXTS 1676812 1676812 ONLY_A 0 ONLY_B 0
[user@server package]$ tail -3 ../package.log | sed "s#/ecel/UFAD/c.valentin/tt#~#"
PRECHECK_EXIT=0
PRECHECK_MARKERS=0
PACKAGE_OK ~/flash/runs/test1/package/IHP__RVSoC8787
[user@server package]$

```

Figure 45. The release folder, the shape and label comparison with the signed-off chip, and the end of the package log with IHP's pre-check on the release file.

Saving with a new grid could round a shape or drop a label. So we compared the release file with the signed-off chip shape by shape. The shape comparison found no difference in area. A label-by-label comparison found all 1,676,812 labels the same. The shape log lists eight label layers as differing. Each has 0 area and 0 polygons. IHP's pre-check on the release file found 0 errors.

7.3 Mistakes we made on the chip

What we saw	What was wrong	What caught it
three fix rounds were kept	signoff had stopped early. The script compared the old CHECK.txt with itself	requiring a new CHECK.txt
a hold fix list at the fast corner	it would have made slow setup -2.65 ns	testing every list at all three corners
a noise fix	it made slow setup -0.154 ns. Fix commands ignore setup unless given a limit	the same test
buffers read 0.14 pF	PrimeTime guessed a wire for cells not yet placed	Fusion Compiler's real placement and routing
a good round was undone	an edited checker made old records look stale	a check-only run first
a queue waited for hours	its wait test matched its own start command	watching the log
Formality reported 5 unmatched points	the router's antenna diodes. They have no logic	removing them from the Formality copy only
139 of 140, with ATPG "absent"	check.py wanted a result row named atpg that nothing writes. The ATPG verdict is atpg_coverage. It passed	reading check.py

7.4 Status of the chip

The fresh build's final checker report says SIGNOFF CLEAN. All 139 required checks pass. They include timing at every corner in both modes. Setup has at least +0.707 ns to spare. Hold has at least +0.116 ns. The checks also include IHP's DRC rules with the density rules and IHP's pre-check. LVS runs with the memory and pad cells boxed. Formality and the scan test are among the 139 too. The scan test covers 96.56% of the modeled stuck-at faults. Separate reports outside those 139 checks cover the rest. Every test program passes on the gate netlist with real delays at all three corners. SpyGlass finds every input from the board synchronized. The worst static supply drop is 2.3%, or about 2.4% with the package. Both are well inside our 10% allowance. The release package holds the exact layout we checked. We saved it in the form IHP asks for. IHP's own pre-check on it is clean. We have a working chip design. It passes every check described here, and its release package is ready to submit to IHP. We will know whether the built chip works only after the factory makes it and someone runs the bring-up test below.

Three gaps remain. We have no worst and best wire models qualified by the factory. LVS has not checked the insides of the boxed memory and pad cells. We have no qualified current models for dynamic voltage-drop checks. Sections 6.1 and 6.6 test the first and third gaps another way. Nothing in this document checks the insides of the boxed cells. These checks do not replace the factory's own models or its acceptance of a submitted design.

Some checks are narrower than their names. The scan test finds only stuck-at faults and misses faults that only make a path slow. The EM check covers only the steady current in the power grid. SpyGlass checked the RTL without the pad ring, so the final netlist has no CDC check. The memory test leaves out the stack area of data memory.

One more thing to expect. IHP's own checks on a submitted design run LVS without boxing the memory and pad cells. Those cells fail IHP's LVS rules even alone. IHP's report may therefore flag them. Such a flag would be about IHP's cells. Our boxed LVS already checks our wiring to them.

Bring-up is the first test of a built chip. The package's datasheet is `doc/Datasheet.md` in the release folder. It gives these steps:

6. Tie `test_mode`, `scan_en` and `scan_in` low.
7. Apply 1.2 V and 3.3 V. Start a 50 MHz clock.
8. Hold reset low for at least 5 clock cycles. Then raise reset and `load_en` together.
9. Load the program through the serial pins. Send each word as a 42-bit frame on `sdi` with the most significant bit first. The frame is a 10-bit word address followed by 32 bits of data. The chip samples `sdi` on each rising edge of `sc1k`. Keep each half period of `sc1k` at least 4 chip clock cycles. Follow the rule for `sdi` in section 6.5.
10. Lower `load_en`. The processor starts the program.
11. Read the output pins. 0x80 means the program passed.

One-page summary

The block commands below show how our block run was driven, with an example run folder called runs/my_ibex. The chip commands use the repository and work as written.

```
cd ~/flash
mkdir -p runs/my_ibex          # put run_cfg.tcl in it and set OUT to this folder

# Stages 1 and 2 in Fusion Compiler          check after each stage
~/flash/flow/run_fc.sh runs/my_ibex
  setup      read the code, corners, timing rules      FLASH_STAGE_DONE setup
  synth      compile_fusion -to logic_opto             rpt/area_synth.rpt
  floorplan  size the block, place the pins            FLASH_UNPLACED_PORTS 0
  pg         ring, mesh, straps, rails                rpt/pg_conn_pg.rpt: 0 floating wires
  place      compile_fusion -from initial_place        rpt/legality_place.rpt
  cts        clock tree, decaps                       rpt/clock_qor.rpt
  route      route_auto, route_opt, cleanup           rpt/route_cleanup_1.rpt
  final      fillers, checks, write the files          rpt/check_routes.rpt
  it worked only if the log has FLASH_FLOW_DONE

# Stage 3 signoff
~/flash/flow/signoff.sh runs/my_ibex
  StarRC, PrimeTime at 3 corners, Formality, PrimePower, DRC, LVS
  answer in signoff/CHECK.txt

# Fix loop
E=~/.flash/flow/eco_fixed.sh
$E runs/my_ibex slow setup
$E runs/my_ibex slow drc          # repeat until FLASH_ECO_NONE
# try a hand list in my_changes.tcl at each corner before you apply it
TRY_ONLY=1 MANUAL_CHANGES=my_changes.tcl $E runs/my_ibex slow manual
MANUAL_CHANGES=my_changes.tcl $E runs/my_ibex slow manual
```

```
# Stages 5 to 7 on the IHP chip, from the repository
git clone https://github.com/XxMasterepicxX/ibex-sg13g2-chip.git ~/flash
bash ~/flash/setup/setup.sh      once
bash ~/flash/chip/make_chip.sh my_chip  stages 5 and 6 in one command, log in runs/my_chip
  run_fc.sh      synthesis and scan, then floorplan through route
  finish.sh      bond pads, seal ring, fill, chip DRC, IHP pre-check
  signoff.sh     6 PrimeTime runs, Formality, DRC, LVS, chip LVS
  margin_fix.sh  test each change list at every corner before applying it
  gls.sh         programs on the gates at slow, typical, fast
  atpg.sh        scan patterns and both replays
  cdc.sh         clock crossings
  ir/ir.sh       voltage drop and EM, with controls
R=~/.flash/runs/my_chip
python3 ~/flash/chip/review/sign.py $R "Your Name"      after reading the drafts
bash ~/flash/chip/final_signoff.sh $R                  must end with SIGNOFF CLEAN
bash ~/flash/chip/package/package.sh $R                must end with PACKAGE_OK
```

References

#	Source	Used for
1	Miguel, "ASIC Physical Implementation, Student Tutorial: Cadence RTL to GDSII Flow", team document	the stage structure this tutorial follows
2	lowRISC, Ibex RISC-V core, https://github.com/lowRISC/ibex	the design
3	IHP, IHP Open PDK for SG13G2, https://github.com/IHP-GmbH/IHP-Open-PDK	the chip kit with its KLayout DRC and LVS rules and its process specification
4	IHP, Open-Silicon-MPW, https://github.com/IHP-GmbH/Open-Silicon-MPW	the submission folder layout and <code>gen_structure.py</code>
5	PULP Platform, Croc, https://github.com/pulp-platform/croc , and "Croc: An End-to-End Open-Source Extensible RISC-V MCU Platform to Democratize Silicon", arXiv:2502.05090	a RISC-V chip on SG13G2 built around the Ibex fork CVE2
6	"HyperCroc: End-to-End Open-Source RISC-V MCU with a Plug-In Interface for Domain-Specific Accelerators", arXiv:2603.12308	Croc silicon measured at 72 MHz and 1.2 V
7	Synopsys user guides and man pages installed with the tools. They cover Fusion Compiler, PrimeTime, StarRC, Formality, PrimePower, IC Validator, RedHawk-SC, VCS, TestMAX and SpyGlass	every tool command
8	Our block run <code>tutorial1_ibex_saed32</code> , our first chip build <code>ihp_chip_2026_v2_20261002</code> , and the fresh repository build <code>test1</code>	every screenshot and number
9	ibex-sg13g2-chip, https://github.com/XxMasterepicxX/ibex-sg13g2-chip	the chip's scripts in stages 5 to 7